

**Estudio experimental sobre el impacto del uso de SYCL en el cálculo de
Streamlines, Streaklines y Pathlines**

Juan Pablo Cerinza Zaraza

Ashley Michelle Calderon Villamizar

Trabajo de grado para optar el título de Ingeniero de Sistemas

Director

Sergio Augusto Gélvez Cortés

PhD en Ciencias de la Computación

Universidad Industrial de Santander

Facultad De Ingenierías Fisicomecánicas

Escuela De Ingeniería De Sistemas e Informática

Pregrado en Ingeniería de Sistemas e Informática Bucaramanga

2026

Dedicatoria

Este proyecto va dedicado a todas las personas que me apoyaron y aportaron a mi desarrollo como persona, y como profesional. A todas las personas que quiero.

A mis padres.

A mi hermana.

A mis amigos.

A Daniel.

Contenido

Introducción	3
1. Planteamiento del problema	4
2. Objetivos	6
2.1. Objetivo General	6
2.2. Objetivos Específicos	6
3. Estado del Arte	6
3.1. Computación Autonómica	7
3.1.1. MAPE-K	8
3.1.2. Mecanismos de Descripción	9
3.1.3. Mecanismos de Adaptación	10
3.2. Sistemas IoT Autonómicos	12
3.2.1. Toma de Decisiones en Sistemas IoT Autonómicos	12
4. Marco Teórico	13
4.1. COMPUTACIÓN DE ALTO RENDIMIENTO (HPC)	13
4.2. COMPUTACIÓN HETEROGÉNEA	14
4.3. VISUALIZACIÓN DE CAMPOS DE FLUJO	15
4.4. ALGORITMOS DE INTEGRACION NUMERICA (RK4)	17
4.5. MODELOS DE PROGRAMACION EN PARALELO	18
4.6. SYCL (Y ESTANDARES DE PORTABILIDAD)	19
5. Metodología de la Investigación	20
5.1. Implementación de RK4	21
5.2. Desarrollo del ambiente de pruebas	21

5.3. Recopilación de resultados	22
6. Algoritmo de visualización con RK4	23
6.1. RK4 implementado en CUDA	23
6.2. Migración del código de CUDA	24
7. Estado Actual Frente al de Referencia	27
7.1. Conociendo el estado actual	27
7.2. Centralizando los Datos	29
7.3. Implementado un Event-Handler	31
7.4. Comparando Arquitecturas	33
8. Adaptando la Arquitectura	36
8.1. Identificando Los Problemas, Estableciendo Acciones	36
8.2. De Acciones, a Instrucciones	38
8.3. De Instrucciones a Acciones	41
9. Validación de la Implementación	44
9.1. Escenario Experimental A	45
9.2. Escenario Experimental B	46
10. Análisis de Resultados	48
10.1. Resultados De Escenario Experimental A	48
10.2. Resultados De Escenario Experimental B	52
11. Conclusiones y Trabajo Futuro	57
Referencias	59

Lista de figuras

1.	El ciclo auto-adaptativo MAPE-K (Gorla, Pezzè, Wuttke, Mariani, y Pastore, 2010)	8
2.	Arquitectura del prototipo de Smart Campus definido por	28
3.	Arquitectura actual del proyecto con el agregador	30
4.	Diagrama de flujo del proceso realizado por Lexical actualizado	31
5.	Primera propuesta del proceso a realizar por <i>Looker</i>	32
6.	Proceso realizado por el observador durante el consumo de los mensajes enviados por los dispositivos	33
7.	Proceso reloj realizado por el observador para la actualización del estado de la aplicación	34
8.	Arquitectura actual del proyecto	36
9.	Proceso para la identificación de problemas de los requerimientos de datos	37
10.	UML de la estructura de directivas y ordenes	39
11.	Proceso de decisión de acciones base realizado por Bran	40
12.	Proceso base de DoThing	42
13.	Proceso para la ejecución de ordenes de adición	42
14.	Proceso para la ejecución de ordenes de reinicio	43
15.	Proceso para la ejecución de ordenes de reconfiguración	43
16.	Arquitectura completa del proyecto	44
17.	Diagrama del escenario experimental A	45
18.	Validación de la declaración de referencia realizada para el escenario experimental A	46
19.	Aplicaciones definidas para el desarrollo del escenario experimental B	47
20.	Aplicación y directivas registradas en Bran	48
21.	Looker evalúa el estado de la aplicación	49

22.	Bran identifica los problemas y establece las acciones	49
23.	DoThing ejecuta las acciones establecidas por Bran	50
25.	Bran identifica otros problemas en la aplicación	50
24.	Contenedores creados por DoThing en respuesta a las órdenes recibidas . . .	51
26.	Bran establece acciones de reconfiguración para adaptar la aplicación	52
27.	Aplicaciones y directivas registradas en Bran	53
28.	Dos instancias de looker monitoreando las aplicaciones declaradas	54
29.	Se matan los contenedores con los servicios iniciales	54
30.	Bran intenta reiniciar los contenedores	55
31.	DoThing no puede encontrar los contenedores	55
32.	Bran crea órdenes de adicción	56

Lista de tablas

1.	Parámetros de entrada del ejecutable	24
----	--	----

Lista de apéndices

1.	Directivas declaradas la aplicación Malaga	64
2.	Directivas de la aplicación Malaga	65
3.	YAML de la aplicación Patio	67
4.	YAML la aplicación House	67
5.	Directivas de la aplicación Patio	68
6.	Directivas de la aplicación House	69

Glosario

IoT: Internet of Things

LDA: Lenguaje de Descripción de Arquitectura (Architecture Description Language)

Resumen

Título: Estudio experimental sobre el impacto del uso de SYCL en el cálculo de Streamlines, Streaklines y Pathlines¹

Autor: Juan Pablo Cerinza Zaraza²

Palabras clave: Computación Autonomica, Internet de las cosas, Adaptabilidad, Arquitecturas Software, Lenguajes de Descripción de Arquitecturas, Smart Campus

Descripción:

La complejidad de los sistemas computacionales tiene origen en diversos factores. El aumento de la cantidad de dispositivos que los componen; junto con las condiciones variables de sus entornos de ejecución; dificultan la administración de los sistemas computacionales. Algunos de los sistemas que se ven afectados por esta complejidad, suelen ser los sistemas IoT, entre ellos los conocidos como Smart Campus. En este trabajo de investigación, se propone una implementación, para la plataforma IoT Smart Campus UIS, basada en los principios de computación autónoma, con el fin de reducir la complejidad de la administración de las aplicaciones desarrollándose sobre esta. Para lograrlo, se inicia definiendo una notación que permite la declaración de las arquitecturas objetivo; segundo, se diseña e implementa un mecanismo responsable del monitoreo de la plataforma al igual una manera de establecer las diferencias entre el estado objetivo y el de referencia; Posteriormente, se implementa una serie de mecanismos para modificar el estado actual de la arquitectura; finalmente, se lleva a cabo la validación de la implementación para evaluar su capacidad para reducir las diferencias entre el estado objetivo y el estado presente.

¹Trabajo de Investigación

²Facultad De Ingenierías Fisicomecánicas. Escuela De Ingeniería De Sistemas e Informática.
Director: PhD. Sergio Augusto Gélvez Cortés

Abstract

Title: Self-adaptive software architecture mechanisms for the Smart Campus UIS platform³

Autor: Juan Pablo Cerinza Zaraza⁴

Palabras clave: Autonomic Computing, Internet of Things, Adaptability, Software Architecture, Architecture Description Languages, Smart Campus

Descripción:

The complexity of computational systems originates from various factors. The increase in the number of devices that compose them, along with the variable conditions of their execution environments, makes the administration of computational systems challenging. Some of the systems affected by this complexity are often IoT systems, including those known as Smart Campus. In this research work, an implementation is proposed for the IoT Smart Campus UIS platform based on the principles of autonomic computing, aiming to reduce the complexity of managing applications developed on it. To achieve this, it begins by defining a notation that allows the declaration of target architectures. Second, a mechanism for monitoring the platform is designed and implemented, along with a way to establish differences between the target state and the reference state. Subsequently, a series of mechanisms are implemented to modify the current state of the architecture. Finally, the implementation is validated to assess its ability to reduce differences between the target state and the present state.

³Research Work

⁴Faculty of Physics-Mechanics Engineering. School of Systems Engineering and Computer Science.
Advisor: PhD. Sergio Augusto Gélvez Cortés

Introducción

El desarrollo de la computación de alto rendimiento (High Performance Computing, HPC) ha permitido abordar problemas científicos y de ingeniería que requieren un procesamiento intensivo de datos. Dentro de este ámbito, la programación paralela constituye una estrategia esencial para aprovechar de manera eficiente las arquitecturas modernas de cómputo, particularmente aquellas basadas en unidades de procesamiento gráfico (GPU). Este enfoque resulta especialmente relevante en áreas como la dinámica de fluidos computacional (CFD, por sus siglas en inglés), en la cual el cálculo de trayectorias de partículas (representadas mediante Streamlines, Streaklines y Pathlines) demanda una elevada capacidad de cómputo y optimización en la gestión de recursos.

Entre las tecnologías predominantes en este campo, CUDA (Compute Unified Device Architecture) se ha consolidado como una herramienta de referencia debido a su estrecha integración con hardware de NVIDIA y a la madurez de su ecosistema de desarrollo. No obstante, la dependencia de un único proveedor limita su portabilidad y dificulta la adaptación a entornos heterogéneos. En respuesta a esta necesidad, ha emergido SYCL, un estándar desarrollado por el consorcio Khronos, que ofrece un modelo de programación en C++ para la ejecución paralela en dispositivos diversos (CPU, GPU, FPGA, entre otros). SYCL promueve la portabilidad y la interoperabilidad, características de creciente relevancia en la investigación y la industria, al tiempo que busca mantener un alto nivel de rendimiento.

El presente trabajo tiene como propósito realizar un estudio experimental sobre el impacto del uso de SYCL en el cálculo de Streamlines, Streaklines y Pathlines, comparando su rendimiento con una implementación equivalente desarrollada en CUDA. El análisis se centrará en métricas de tiempo de ejecución, utilización de memoria y eficiencia de cómputo, con el fin de establecer una comparación objetiva entre ambos enfoques. Para garantizar condiciones homogéneas, se emplearán herramientas de perfilamiento como NVIDIA Nsight Systems/Compute y entornos controlados mediante Docker, lo que permitirá asegurar la

reproducibilidad de los experimentos.

Con ello, la investigación busca contribuir al conocimiento sobre la aplicabilidad de modelos de programación portables en escenarios de alto costo computacional, aportando evidencia que permita valorar la pertinencia de SYCL como alternativa a CUDA en el ámbito de la dinámica de fluidos y, en general, en el desarrollo de software científico de alto rendimiento.

1 Planteamiento del problema

La visualización de campos de flujo, mediante técnicas como Streamlines, Streaklines y Pathlines, es fundamental en diversas áreas de la ciencia e ingeniería, incluyendo la dinámica de fluidos computacional (CFD), la meteorología, y el modelado de procesos físicos (Camp, Garth, Childs, Pugmire, y Joy, 2011). Estos métodos permiten representar de forma intuitiva el comportamiento de partículas en movimiento dentro de un campo vectorial, lo cual facilita el análisis y la interpretación de fenómenos complejos.

Sin embargo, la simulación precisa y eficiente de estas trayectorias implica un alto costo computacional, especialmente cuando se trabaja con volúmenes de datos grandes o se requieren resoluciones espaciales y temporales finas (Pugmire, Childs, Garth, Ahern, y Weber, 2009). En este contexto, la programación tradicional en C/C++, aunque eficiente, puede resultar insuficiente para explotar de manera óptima las capacidades de cómputo de los sistemas modernos, en especial aquellos que cuentan con aceleradores como GPUs.

En respuesta a esta necesidad, han surgido enfoques como la programación heterogénea, que busca distribuir la carga computacional entre distintos dispositivos de procesamiento, permitiendo una ejecución más rápida y eficiente. En particular, el estándar SYCL, basado en C/C++, ofrece una solución portable y flexible para desarrollar aplicaciones que aprovechen arquitecturas heterogéneas sin perder la expresividad y control del lenguaje de bajo nivel (Keryell, 2019).

A pesar de sus beneficios teóricos, existe una brecha de conocimiento práctico respecto al impacto real del uso de SYCL en problemas computacionalmente intensivos como el

cálculo de líneas de flujo. No se cuenta con suficiente evidencia comparativa que cuantifique su desempeño frente a enfoques tradicionales en términos de tiempo de ejecución, consumo de memoria y escalabilidad, con respecto a esta técnica de visualización en particular. Para cuantificar el desempeño se realizan estudios experimentales usando métricas como tiempo de ejecución, uso de memoria y escalabilidad (Camp y cols., 2013), relacionadas con características del problema como: tamaño del dataset, tamaño del conjunto de semillas, distribución del conjunto de semillas y complejidad del campo vectorial (Camp y cols., 2011).

2 Objetivos

2.1 Objetivo General

- Comparar el desempeño computacional de una técnica de visualización de campos de flujo (Streamlines, Streaklines y Pathlines), que involucra el método de Runge-Kutta implementado en lenguaje C/C++, mediante métodos tradicionales de cómputo y programación heterogénea con SYCL, evaluando métricas como el tiempo de ejecución, uso de memoria y escalabilidad.

2.2 Objetivos Específicos

- Formular una versión optimizada del algoritmo RK4 en SYCL, con el propósito de evaluar su desempeño en arquitecturas heterogéneas.
- Comparar el comportamiento de las implementaciones en C/C++ y en SYCL bajo diferentes configuraciones del problema (tamaño del dataset, conjunto de semillas y complejidad del campo vectorial), con el fin de identificar diferencias en su eficiencia y desempeño.
- Analizar comparativamente las ventajas, limitaciones y condiciones de uso de SYCL frente a implementaciones tradicionales en CPU y en GPU con CUDA, considerando métricas de tiempo de ejecución, uso de memoria y escalabilidad.

3 Estado del Arte

Con el objetivo de explorar a fondo el panorama actual de la computación autonómica, y en particular, los mecanismos de adaptación de arquitecturas de software y los requisitos fundamentales para su implementación, se llevó a cabo una revisión de la literatura en diversas bases de datos. Esta revisión abarcó un recorrido que partió de una visión general y se adentró en aspectos cada vez más específicos. Durante este proceso, se examinaron

detalladamente las propuestas y componentes clave de sistemas de software autónomos, así como las diversas nociones relacionadas con notaciones, algoritmos para la comparación de estructuras de datos y los mecanismos esenciales para la adaptación de arquitecturas de software.

3.1 Computación Autónoma

El concepto de computación autónoma, definido inicialmente por IBM (2001), se refiere a un conjunto de características que presenta un sistema computacional el cual le permite actuar de manera autónoma, o auto-gobernarse, con el fin de alcanzar algún objetivo establecido por los administradores del sistema (Lalanda, Diaconescu, y McCann, 2014).

Los 8 elementos clave, definidos por IBM, que deberían presentar este tipo de sistemas son:

1. Auto-conocimiento: habilidad de conocer su estado actual, las interacciones del sistema.
2. Auto-configuración: capacidad de reconfigurarse frente a los constantes cambios en el entorno.
3. Auto-optimización: búsqueda constante de optimizar el funcionamiento de sí mismo.
4. Auto-sanación: aptitud de restaurar el sistema en el caso de que se presenten fallas.
5. Auto-protección: facultad de protegerse a sí mismo de ataques externos.
6. Auto-conciencia: posibilidad de conocer el ambiente en el que el sistema se encuentra.
7. Heterogeneidad: capacidad de interactuar con otros sistemas de manera cooperativa.
8. Abstracción: ocultar la complejidad a los administradores del sistema con objetivos de alto nivel de abstracción.

En el caso de que un sistema tenga una implementación parcial de las características mencionadas, este podría considerarse autónomo. Por ende debería tener la capacidad de

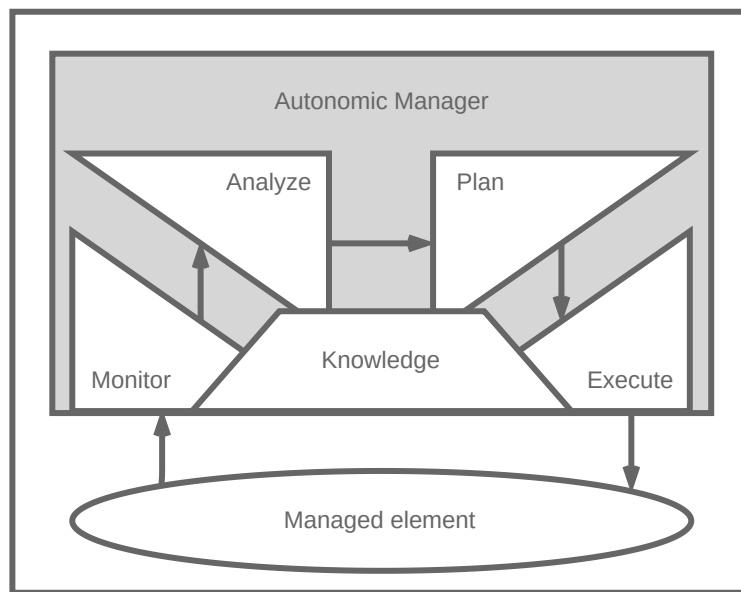
lidar con los problemas como la complejidad, heterogeneidad e incertidumbre (Salehie y Tahvildari, 2005) al igual que reducir la cantidad de recursos tanto técnicos como humanos requeridos para mantener los sistemas en funcionamiento.

3.1.1 MAPE-K

En cuanto a la implementación de las características, IBM propone un modelo de ciclo auto-adaptativo, denominado MAPE-K (Krikava, 2013). Este acercamiento, compuesto de cinco fases, es uno de los ciclos de control más usados en implementaciones de sistemas auto-adaptativos y computación autónoma (Arcaini, Riccobene, y Scandurra, 2015). En la figura 1, se presentan las fases que el *manejador* debe desarrollar para así administrar cada uno de los elementos del sistema computacional fundamentado en una base de conocimiento común (Gorla y cols., 2010).

Figura 1:

El ciclo auto-adaptativo MAPE-K (Gorla y cols., 2010)



Cada una de estas fases son:

- Monitorear (M): Esta fase se compone de la recolección, filtración y reporte de la información adquirida sobre el estado del elemento a manejar.

- Analizar (A): La fase de análisis se encarga de interpretar el entorno en el cual se encuentra, así como predecir posibles situaciones comunes y diagnosticar el estado del sistema.
- Planear (P): Durante la planificación se determinan las acciones a tomar, con el fin de llegar a un objetivo establecido a partir de una serie de reglas o estrategias.
- Ejecutar (E): Finalmente, se ejecuta lo planeado usando los mecanismos disponibles para el manejo del sistema.

Es de resaltar que este modelo, aunque útil para el desarrollo de este tipo de sistemas, es bastante general en cuanto a la estructura y no usan modelos de diseño establecidos (Ouareth, Boulehouache, y Mazouzi, 2018).

3.1.2 Mecanismos de Descripción

La fase de monitoreo dentro del ciclo MAPE-K es vital para el funcionamiento del manejador autónomo, pues es a partir de la información que se construirá la base de conocimiento requerida por las demás partes del ciclo. Parte de esta, está compuesta por el *estado del sistema*, el cual incluye la descripción de este en un momento dado (Weiss, Zeller, y Eilers, 2011).

Existen varias implementaciones de mecanismos de auto-descripción, y la utilidad de cada uno de ellos varía dependiendo en el tipo de sistema de software que se esté usando. Para el marco del proyecto, son relevantes aquellos ,mecanismos que estén orientados a los sistemas embebidos e IoT, algunos de estos son:

- **JSON Messaging:** Iancu y Gatea (2022) plantean un protocolo que emplea mensajería entre *gateways* con el fin de recibir información sobre estas. En términos simples, estas funcionan como un *ping* hacia el nodo que luego retorna sus datos, al igual que los dispositivos conectados a ella, al encargado de recolectar toda esta información con el fin de construir una descripción del sistema.-

- **IoT Service Description Model:** O IoT-LMsDM, es un servicio de descripción desarrollado por Wang, Sun, y Aiello (2021) el cual está orientado al contexto, servicios e interfaz de un sistema IoT. De este se espera poder contar no solo con descripciones del estado del sistema en términos del ambiente, pero la funcionalidad (es decir, los *endpoints* a usar) al igual que las estructuras de datos que estos consumen.
- **Adaptadores de Auto-descripción:** En este acercamiento a los mecanismos de auto-descripción, se tienen adaptadores los cuales emplean los datos generados por los sensores del componente gestionado con el fin de realizar la determinación de la arquitectura desplegada. De igual manera, este acercamiento permite realizar modificaciones a la descripción de manera manual en caso de que se detecten problemas (Jiménez Herrera, 2022).

De esto podemos ver no solo las diferentes maneras en las que las implementaciones realizan las descripciones de los sistemas asociados, sino que también el alcance de estos en cuanto a lo que pueden describir.

3.1.3 Mecanismos de Adaptación

La adaptación, en el contexto de la computación autónoma, es la parte más importante en cuanto a la auto-gestión de un sistema de software se refiere. Así mismo, presenta el mayor reto debido a la necesidad de modificar código de bajo nivel, tener que afrontarse a incertidumbre de los efectos que pueden tener dichas alteraciones al sistema al igual que lidiar con esto en *runtime* debido a los problemas que el *downtime* tendría en los negocios (Lalanda y cols., 2014).

Esta adaptabilidad puede exponerse en múltiples puntos dentro de un sistema de software. Pueden realizarse adaptaciones a nivel de kernel, lenguaje de programación, arquitectura e incluso datos (Lalanda y cols., 2014).

Manteniéndose en el marco del proyecto, son relevantes aquellas soluciones relacionadas con la modificación de la arquitectura. Siendo así, se consideraron únicamente los mecanismos

de adaptación de componentes, o de reconfiguración:

- **Binding Modification:** Este mecanismo hace referencia a la alteración de los vínculos entre los diferentes componentes de la arquitectura. Estos tienen el objetivo de modificar la interacción entre componentes, lo que es especialmente común en implementaciones con *proxies*. Este tipo de mecanismo de adaptación fue usado por Kabashkin (2017) para añadir fiabilidad a la red de comunicación aérea.
- **Interface Modification:** Las interfases funcionan como los puntos de comunicación entre los diferentes elementos de la arquitectura. Siendo así, es posible que la modificación de estos sea de interés con el fin de alterar el comportamiento de un sistema al igual que soportar su heterogeneidad. Esto puede observarse en el trabajo desarrollado por Liu, Parashar, y Hariri (2004) en donde se define la utilidad de dichas adaptaciones al igual que la implementación de las mismas.
- **Component Replacement, Addition and Subtraction:** En términos simples, este mecanismo se encarga de alterar los componentes que hacen parte de la arquitectura; de esta manera, modificando su comportamiento. Ejemplos de lo anterior pueden verse en el trabajo de Huynh (2019), en el cual se evalúan varios acercamientos a la reconfiguración de arquitecturas a partir del remplazo de componentes tanto a nivel individual al igual que grupal.

Este acercamiento a la mutación de la arquitectura también puede verse en el despliegue de componentes en respuesta a cambios en los objetivos de negocio de las aplicaciones, así como reacción a eventos inesperados dentro de la aplicación. Esto puede verse en trabajos como el de Patouni y Alonistioti (2006) donde se realizan este tipo de implementaciones.

3.2 Sistemas IoT Autonomicos

Partiendo de lo anteriormente establecido, una de las áreas en las que estos conceptos de computación autónoma se ha hecho presente es el campo del IoT. Los acercamientos entre estas dos ramas de las ciencias de la computación se ha venido presentado con el objetivo de dar a los sistemas IoT la capacidad de adaptarse a su ambiente, teniendo el fin de optimizar los recursos disponibles y reducir la necesidad de interacción humana, a partir de la automatización de la configuración y procesos para mantener la disponibilidad de los servicios (Ashraf, Tahir, Habaebi, y Isoaho, 2023).

Ejemplos de estas implementaciones de propiedades autónomas en sistemas IoT pueden apreciarse en trabajos como el de Rajan, Balamuralidhar, Chethan, y Swarnahpriyaah (2011), donde después de trabajar con diferentes protocolos de manejo de redes con el fin de poder realizar una administración de una red de sensores, se determinó que el manejo de un sistema de tal magnitud sería la limitante principal en el crecimiento del mismo. Esto terminó en la creación de un *control loop*, el cual se encargaba de la configuración automática de la red de sensores a partir de parámetros ambientales y objetivos de calidad de servicio.

3.2.1 Toma de Decisiones en Sistemas IoT Autonomicos

Algo a resaltar es el tipo de métricas con las cuales se determina la validez. Estas varían dependiendo de las necesidades de la implementación al igual que los objetivos definidos por los administradores del sistema. Durán-Polanco y Siller (2023), identificaron cuatro tipos de maneras de tomar decisiones sobre un sistema IoT.

- **Criterion-Driven:** Orientado principalmente al uso de algoritmos con el fin de establecer el camino a seguir en la toma de decisiones. Este acercamiento puede tener asociado un modelo matemático, algoritmos genéticos, Q-Learning, etc. dependiendo de la complejidad del sistema.
- **Data-Driven:** En este, a partir de datos de entrada y salida definidos, se busca deter-

minar cómo generar datos de nuevo salida definidos dadas unas entradas. Estas están se usan principalmente en implementaciones que usan aprendizaje supervisado o sin supervisión.

- **Utility-Driven:** En el caso de la utilidad, se refiere a la comparación y evaluación de diferentes alternativas de modelos matemáticos con el fin de establecer una mejor toma de decisiones. Este está relacionado con algunos conceptos usados en teoría de juegos y modelos multicriterio.
- **Probability-Driven:** Este hace uso de modelos probabilísticos, tales como estocásticos o bayesianos, para la toma de decisiones. Normalmente se emplean para aprendizaje no supervisado para la determinación de parámetros a partir de un punto común dados unos datos.

4 Marco Teórico

4.1 COMPUTACIÓN DE ALTO RENDIMIENTO (HPC)

La computación de alto rendimiento, conocida como HPC, forma parte indisoluble del ámbito de la computación y está dedicada fundamentalmente a la resolución de problemas, que requieren para su resolución un gran poder computacional. El objetivo de HPC es el de aprovechar arquitecturas avanzadas y paralelas que puedan ser utilizadas para resolver tareas que de ser necesarias en sistemas convencionales no se podrían realizar precisamente debido, entre otras consideraciones, a la complejidad de los cálculos a realizar y a la gran cantidad de datos a procesar. (Eijkhout, 2022)

De un modo general, HPC ha pasado a ser parte fundamental de la computación en diversas disciplinas científicas y de la ingeniería, permitiendo realizar simulaciones numéricas, el procesamiento de grandes cantidades de datos o el modelado detallado de fenómenos físicos. Este enfoque ha permitido la expansión de áreas como la predicción del tiempo (clima), la exploración espacial, la biología computacional, el diseño de nuevos materiales, etc.

A lo largo de la historia (de los sistemas HPC) las supercomputadoras se transforman en infraestructuras distribuidas y clústeres HPC y estos cambios han conducido también a la aparición de paradigmas de programación que permiten explotar el paralelismo como por ejemplo MPI (Message Passing Interface) y OpenMP (Open Multi-Processing) y más recientemente la aparición de las GPUs han permitido incrementar todavía más la capacidad computacional por medio de un paralelismo masivo orientado hacia cálculos científicos y simulaciones.(Eijkhout, 2022)

Un aspecto clave de los sistemas HPC en la actualidad es la migración hacia arquitecturas heterogéneas que integran diferentes tipos de procesadores (CPU, GPU, FPGA y aceleradores específicos) con vista a una mayor eficiencia. El éxito que han ido logrando estas arquitecturas no responde solamente a la necesidad de un mayor poder computacional, sino que tiene que ver también en el avance del conocimiento de la factibilidad y del consumo energético, así como de la escalabilidad. En este marco, la programación y la portabilidad del código, se convierte en uno de los pilares que conduce a los avances que presentan nuevos modelos como SYCL, que intenta crear un acceso unificado a las diferentes plataformas, conservando al mismo tiempo el rendimiento (Reinders y cols., 2021) .

Por consiguiente, HPC no debe ser entendida solo como la existencia y uso de supercomputadoras, sino que forma parte de un conjunto de tipos de tecnología que siempre están evolucionando y que tienen mucha repercusión en la aceleración del conocimiento científico y la posibilidad de poder tratar la solución de problemas que tienen mucho significado a nivel mundial.

4.2 COMPUTACIÓN HETEROGÉNEA

La computación heterogénea tiene su fundamento en el uso conjunto de varios tipos de procesadores existentes en un mismo sistema y tiene como objetivo el aumento del rendimiento y de la eficiencia. En este modelo de computación no solo se hace funcionar la CPU, sino que se hacen funcionar otros dispositivos adicionales a esta como las GPU, FPGA u otros

tipos de aceleradores, de forma que se distribuyen las tareas en función de las capacidades de cada una de las unidades de procesamiento.

El interés por este modelo ha crecido en los últimos años debido a que los problemas actuales no tienen únicamente que ver con la capacidad de cómputo, sino también con la eficiencia energética y la escalabilidad. Si bien las CPU ofrecen flexibilidad y son apropiadas para las tareas secuenciales, las GPU llegan a aportar niveles de paralelismo muy elevados, de tal forma que se pueden utilizar en simulaciones, cálculos numéricos masivos o grandes volúmenes de datos. (Mittal y Vetter, 2015).

La adopción de arquitecturas heterogéneas se ha visto favorecida por la aparición de entornos de programación como CUDA, OpenCL y SYCL, que facilitan su programación y proporcionan diferentes niveles de portabilidad. (Keryell, 2019). Aun así, la computación heterogénea tiene también sus inconvenientes, como la necesidad de encontrar algoritmos a partir de los cuales las computaciones puedan adaptarse a hardware heterogéneo y la gestión de la carga de trabajo en los distintos dispositivos.

Este paradigma se ha instalado como una de las principales tendencias en la computación de alto rendimiento, debido a que permanece en la línea que se busca entre velocidad de ejecución, eficiencia energética y uso de recursos.

4.3 VISUALIZACIÓN DE CAMPOS DE FLUJO

La visualización de los campos de flujo es una metodología para realizar una representación gráfica del comportamiento de los sistemas dinámicos que se desarrollan con respecto al tiempo y al espacio. Un campo de flujo es la descripción de cómo varía una magnitud vectorial, como puede ser, por ejemplo, la velocidad o la fuerza, en los distintos puntos de un dominio, siendo su análisis importante en la dinámica de fluidos, la meteorología o bien en la física computacional. En este sentido, un campo vectorial puede considerarse como una función que asigna un vector, como podría ser un vector de velocidad, a cada punto del espacio, indicando la dirección y la intensidad del movimiento en un cierto punto; partiendo

de esta idea, es posible definir las distintas maneras de representar de una forma u otra el flujo:

1. Streamlines (Líneas de corriente): curvas que son tangentes al campo de velocidad en un instante dado, indicando la dirección del flujo de forma puntual.
2. Pathlines (Trayectorias): caminos seguidos por las partículas en el tiempo, mostrando su verdadero movimiento.
3. Streaklines (Líneas de emisión): conjunto de puntos ocupados por las partículas que en un momento dado han atravesado el punto considerado.

Estas tres definiciones son equivalentes en flujos estacionarios, pero pueden ser diferentes si se lo aplica a los flujos no estacionarios o turbulentos (Hansen y Johnson, 2005). Un flujo estacionario es aquel en el que el estado del campo de velocidad (dirección y magnitud) no cambia con el tiempo, por lo cual el recorrido de las partículas permanece constante y las tres representaciones -streamlines, pathlines y streaklines- son equivalentes; mientras que un flujo no estacionario implica que el campo de velocidad presenta variaciones en el tiempo y cada tipo de línea hace referencia a una faceta del movimiento: las streamlines hacen referencia a la dirección en un instante, las pathlines representan el recorrido real de las partículas y las streaklines proporcionan la posición de las partículas que han pasado por un punto de referencia.

La importancia de este concepto reside en que es capaz de entender fenómenos complejos mediante representaciones visuales que ayudan en la interpretación de los datos numéricos. En problemas como la simulación de fluidos, la dispersión de contaminantes o el modelado atmosférico, las soluciones numéricas en sí mismas son difíciles de interpretar; por ello, se requiere de la visualización para que se puedan ver patrones, trayectorias y estructuras internas. (Laramée y cols., 2004).

Hay muchas aplicaciones de la visualización de campos de flujo. Por ejemplo, se emplea en el diseño aerodinámico de la industria del automóvil y la aeronáutica para pronosticar el clima; también se utiliza para simular la turbulencia o estudiar el movimiento de partículas

en medios físicos. Igualmente, se utiliza para el análisis de modelos en biomedicina. A partir de todas estas aplicaciones, se requiere de métodos computacionales intensivos para poder resolver numéricamente el problema, como para convertir gráfica y visualmente los resultados. (Laramée y cols., 2004) (Post, Vrolijk, Hauser, Laramée, y Doleisch, 2003).

Sin embargo, la visualización de campos de flujo presenta limitaciones debido principalmente al alto coste computacional que se deriva de ciertos aspectos; por ejemplo, el cálculo de trayectorias, la interpolación de datos en malla tridimensional y el renderizado de grandes cantidades de datos requieren de sistemas que puedan ejecutar tareas en paralelo, y de arquitecturas de computación avanzada. (Post y cols., 2003).

Es en este sentido que desde el proyecto se ha considerado este tipo de problemas, ya que el algoritmo para la integración numérica RK4, optimizado y adaptado a arquitecturas heterogéneas, sirve para calcular trayectorias para partículas dentro de campos de flujo. Por lo que para la visualización no solo se quedaría como una aplicación representativa, sino que también serviría como un caso de estudio para poder validar la eficiencia y portabilidad de las implementaciones en SYCL frente a otras soluciones de forma tradicional.

4.4 ALGORITMOS DE INTEGRACION NUMERICA (RK4)

Los algoritmos de integración numérica constituyen un conjunto de métodos que se idearon para aproximar la solución de Ecuaciones Diferenciales Ordinarias (EDO), sobre todo, cuando no se puede obtener una solución analítica, exacta. Entre todos estos métodos, el método de Runge-Kutta de cuarto orden (RK4) es uno de los más utilizados a causa de su equilibrio entre sencillez, precisión y coste computacional. (Butcher, 2008).

La razón por la que este método es importante, radica en su capacidad para proporcionar resultados con un error relativamente pequeño sin tener que aumentar excesivamente el coste de cálculo. Por eso, el RK4 ha pasado a ser considerado un estándar en lo que se refiere a simulaciones físicas, ingeniería, biología computacional y en general cualquier disciplina que busque modelar fenómenos dinámicos mediante EDO. (Süli y Mayers, 2003).

Entre sus aplicaciones se encuentran la simulación de trayectorias de partículas en campos vectoriales, la dinámica de fluidos, la modelación de circuitos eléctricos, el análisis orbital en astrofísica y la biomecánica. En estas aplicaciones, el RK4 ofrece aproximaciones estables y confiables incluso en sistemas fuertemente no lineales.

Sin embargo, el método tiene una serie de limitaciones ligadas al coste en recursos computacionales, sobre todo cuando la dimensión de los datasets aumenta o cuando se busca una alta resolución tanto temporal como espacial. En estos casos, llevar a cabo el algoritmo de forma secuencial puede acabar convirtiéndose en un cuello de botella, lo que requiere recurrir a utilizar técnicas de paralelización y arquitecturas de computación más avanzadas. (Camp y cols., 2011)(Pugmire y cols., 2009).

En el caso del proyecto, RK4 se convierte en el núcleo matemático que se desea optimizar. Su implementación en un entorno heterogéneo gracias a SYCL permite trabajar en conseguir mejoras en cuanto a el rendimiento y la portabilidad al comparar sus resultados a implementaciones tradicionales en CPU y GPU, como las que basan en CUDA.

4.5 MODELOS DE PROGRAMACION EN PARALELO

La programación paralela es un paradigma que intenta ejecutar muchas operaciones a la vez, dividiendo un problema en subproblemas más pequeños que pueden resolverse de manera simultánea en diferentes unidades de procesamiento. Este modelo, a diferencia de la programación secuencial, trata de maximizar el uso de los recursos de hardware y es fundamental para hacer frente a nuevos problemas de gran escala en ciencia e ingeniería. (Grama, 2011).

Una vez más, su importancia reside en que esas capacidades del hardware moderno están conformadas por procesadores multinúcleo, tarjetas gráficas (GPU) y aceleradores. Es con la ejecución paralela que se puede mejorar los tiempos de cómputo y mejorar la eficiencia energética para las aplicaciones que requieren grandes volúmenes de cálculo, como es el caso de la simulación científica, el aprendizaje automático o la visualización de datos.

Existen diferentes modelos y herramientas de este paradigma. CUDA, desarrollado por NVIDIA, se ha convertido en un estándar de programación para GPU propietarias. OpenCL es un marco de programación abierto que permite usar arquitecturas heterogéneas. OpenMP permite la programación paralela en entornos de memoria compartida para CPU multinúcleo, y por último SYCL, promovido por Khronos Group, extiende el modelo de C++ para múltiples arquitecturas, pero en un marco portable y estándar. (The Khronos Group Inc., 2025)(Reinders y cols., 2021).

No obstante, este método también conlleva ciertas limitaciones. Entre ellas están la dependencia del hardware (hay modelos que se ejecutan solamente en arquitecturas específicas), la complejidad en el control del paralelismo y dependencias como la dificultad de portar aplicaciones entre plataformas heterogéneas. Este tipo de restricciones ha sido el impulso para el desarrollo de otros métodos que consideren una buena relación entre el rendimiento y la portabilidad.

En la parte asociada a este proyecto, la paralelización de las aplicaciones se relaciona estrictamente con la elección de SYCL como modelo de referencia. Esta estrategia permite escribir código portable en C++ y ejecutarlo en CPU, GPU y FPGA, lo que resulta un aspecto fundamental a considerar para evaluar eficiencia y escalabilidad frente a otros modelos convencionales.

4.6 SYCL (Y ESTANDARES DE PORTABILIDAD)

SYCL (del inglés Standard C++ for Heterogeneous Computing) es un estándar abierto de Khronos Group destinado a la extensión del lenguaje C++ con el fin de facilitar la programación en arquitecturas heterogéneas. A diferencia de los modelos basados en implementaciones propietarias, SYCL desarrolla una interfaz de programación de alto nivel que permite a los desarrolladores escribir código portable de forma única para ejecutarse posteriormente en CPU, GPU, FPGA u otros aceleradores (The Khronos Group Inc., 2025).

La importancia del estándar SYCL se encuentra en su desarrollada portabilidad y además

por su productividad. Por enlazarse de manera nativa con C++, permite a los programadores aplicar las características modernas de C++ sin tener que aprender un nuevo paradigma. Este mismo hecho ayuda a promover la interoperabilidad entre familias de plataformas que es especialmente importante en un contexto donde el hardware en la arquitectura puede variar entre diferentes generaciones y fabricantes (Reinders y cols., 2021).

Entre sus aplicaciones más destacadas nos encontramos con proyectos de cómputo de alto rendimiento (HPC), simulaciones científicas, inteligencia artificial y para análisis de datos. Frameworks como oneAPI de Intel, DPC++, o hipSYCL, hacen de SYCL una buena base para ofrecer entornos de desarrollo con rendimiento y a la vez portabilidad, corroborando así su importancia en el marco de estándares modernos de programación paralela (Intel Technologies, 2025)(Alpay y Heuveline, 2021).

No obstante, SYCL también posee algunas limitaciones. La principal limitación radica en su madurez relativa comparada con los modelos ya consolidados, como CUDA, que deriva en un ecosistema de herramientas y bibliotecas todavía no desarrollado. El soporte de hardware tampoco es homogéneo, depende del compromiso que asuman cada uno de los fabricantes, lo que puede hacer que existan diferencias de rendimiento entre dispositivos.

Durante el transcurso de este proyecto, SYCL se establece como la herramienta relevante para comprobar la portabilidad y el rendimiento de la implementación del algoritmo RK4 aplicado a la visualización de campos de flujo. Su uso permite no solo explorar las ventajas de trabajar con un estándar abierto también permite establecer comparativas respecto a otros modelos de tipo propietario en casos de heterogeneidad.

5 Metodología de la Investigación

Para el desarrollo del trabajo de grado, se propuso un plan de 3 fases, que acarrea una secuencia lógica desde la implementación hasta el análisis de los resultados:

5.1 Implementación de RK4

Esta primera fase se basó en el análisis de la fuente de código inicial provista por el director, que contiene una implementación del algoritmo RK4 para la visualización de Streamlines, Streaklines y Pathlines usando CUDA. Posterior a ello, la base de código fue modificada para facilitar su estudio y migración a demás plataformas que usan SYCL o HIP para su estudio en fases próximas.

Actividades

1. Recopilación de datos de entrada de tamaño variable para pruebas.
2. Implementación del algoritmo RK4 en CUDA.
3. Implementación del algoritmo RK4 en SYCL.

Productos

1. Implementaciones del algoritmo RK4 para visualización de Streamlines, Streaklines y Pathlines en los siguientes estándares: CUDA, HIP, oneAPI (SYCL^{*5}) y SYCL
2. Evaluación de los posibles lenguajes de programación para la implementación.

5.2 Desarrollo del ambiente de pruebas

La segunda fase consistió en la definición del cómo se realiza la declaración de la arquitectura. Partiendo de los criterios de selección establecidos en la fase 1, se determinó un lenguaje de notación el cual permitió definir la arquitectura objetivo a alcanzar, al igual que la gramática correspondiente para poder realizar dicha declaración.

La segunda fase consistió de la creación de ambientes controlados donde se ejecutarán los algoritmos, manteniendo las mismas condiciones de cómputo para los algoritmos, y se recopilarán las métricas establecidas para la comparación. Donde se usó Docker y la misma

⁵Denominación que otorga Intel a su uso de SYCL, pues es una extensión del mismo

imagen base de Ubuntu para instalar las dependencias de cada uno de los estándares a evaluar. Para esto se hizo uso del clúster de SC3 para ejecutar las imágenes.

Actividades

1. Determinación de métricas a evaluar.
2. Implementación en Docker del ambiente de pruebas.
3. Ejecución síncrona de pruebas para ambas implementaciones.

Productos

1. Listado de métricas que se recopilán y almacenarán durante la ejecución de las imágenes de Docker
2. Dockerfiles correspondientes a cada uno de los 4 estándares a evaluar
3. Archivos csv con los datos recopilados de las pruebas

5.3 Recopilación de resultados

Durante la tercera fase del proyecto, se buscó determinar e implementar cómo se realiza la comparación entre el estado de la arquitectura obtenido durante la auto-descripción de la misma y el objetivo establecido. Así mismo, y con el fin de reportar a los administradores de los sistemas, también fue necesario definir los *niveles* de similitud entre las 2 arquitecturas.

Durante la tercera fase y última fase, se compilarán los resultados en un formato adecuado para su análisis, y se retroalimentarán los resultados obtenidos para concluir el trabajo de grado.

Durante la tercera y última fase, se compilaron y organizaron los resultados en formatos csv usando Python, se realizaron gráficas que muestran las diferencias entre los 4 estándares en las métricas obtenidas. De la misma forma, se incluyó el aspecto cualitativo de la dificultad de su uso, dadas las condiciones del hardware y software durante el desarrollo de la investigación como punto de comparación entre los estándares.

Actividades

1. Organización de los resultados obtenidos en gráficos y tablas.
2. Retroalimentación y conclusiones de los resultados obtenidos.

Productos

1. Tablas y gráficos correspondientes a las métricas obtenidas y su comparación para cada estándar.
2. Evaluación de los estándares de acuerdo a los datos cuantitativos y cualitativos obtenidos.

6 Algoritmo de visualización con RK4

6.1 RK4 implementado en CUDA

Para establecer el punto de referencia, partimos de una implementación de RK4, que fue provista por el director del proyecto. Implementada usando CUDA, y librerías auxiliares como thrust, para uso de memoria entre device y host; nrrd, para la lectura del archivo de entrada.

El código realiza 3 tareas fundamentales:

- Procesamiento del archivo de entrada, donde usa HDF5 para tomar los datos del campo y almacenarlos en un arreglo bidimensional, para optimizar la lectura en memoria.
- Kernel RK4, que consta de las operaciones relacionadas con el algoritmo RK4, que son realizadas directamente en el device, y sus resultados, almacenados en memoria del host
- Postprocesamiento para visualización, usa los datos de salida del kernel para producir un modelo 3D en formato VTK, de forma opcional, para que pueda ser visualizado.

Asimismo, el código tiene 3 parámetros de entrada, excluyendo el archivo H5, que serán utilizados posteriormente para la realización de pruebas:

Tabla 1:

Parámetros de entrada del ejecutable

Parámetro	Explicación
num-seeds	Corresponde al número de semillas generadas durante el paso inicial previo al kernel RK4
num-steps	El número de veces o iteraciones de cálculos de pasos de RK4 por los que pasará cada semilla. Haciendo que el número total que se calcula RK4 es num-seeds*num-steps
vtk	un valor numérico para que se guarde el archivo de visualización

No obstante, tras un análisis inicial del código, se optó por intercambiar la librería NRRD por HDF5, como sugerencia del director y autor del código, puesto que HDF5 posee características que se adecúan más a su uso en un contexto de computación heterogénea, como lectura/escritura paralela basada en MPI.

En adición, para la toma de métricas de tiempo de ejecución, se tomó el tiempo que le tomó ejecutar las dos primeras tareas del código, para medir aspectos como la lectura de archivos, escritura a memoria, tiempo de ejecución dentro del kernel y comunicación entre host y device. Tras este cambio, asumimos esta implementación como un punto de referencia, para derivar las implementaciones en los demás estándares a analizar.

6.2 Migración del código de CUDA

Con el fin de comparar los estándares de portabilidad, especialmente si se parte de CUDA, es menester incluir el proceso de migración de una base de código en CUDA. Algunos estándares como el de OneAPI de Intel o HIP de AMD, proveen una herramienta que se encarga de migrar el código en CUDA hacia sus respectivos estándares. En el caso de OneAPI, esta se llama *dpct* (Intel DPC++ Compatibility Tool) y para HIP, se llama *HIPify*.

Para ilustrar la diferencia entre la sintaxis de estos estándares, a continuación un fragmento del código de la función main en su versión de CUDA, DPCT y SYCL nativo:

Versión del fragmento en CUDA:

```
1 thrust::device_vector<integrator_rk4> dintg(num_seeds);
2
3 thrust::tabulate(
4     dintg.begin(), dintg.end(), seed_generator{num_seeds});
5
6 hout_i = thrust::copy(dintg.begin(), dintg.end(), hout_i);
```

Versión del fragmento en HIP:

```
1 thrust::device_vector<integrator_rk4> dintg(num_seeds);
2
3 thrust::tabulate(
4     dintg.begin(), dintg.end(), seed_generator{num_seeds});
5
6 hout_i = thrust::copy(dintg.begin(), dintg.end(), hout_i);
```

Versión del fragmento en SYCL*:

```
1 dpct::device_vector<integrator_rk4> dintg(num_seeds);
2
3 dpct::for_each_index(
4     oneapi::dpl::execution::make_device_policy(q_ct1), dintg.begin
5     (),
6     dintg.end(), seed_generator{num_seeds});
7
8 hout_i = std::copy(oneapi::dpl::execution::make_device_policy(q_ct1
9     ),
10    dintg.begin(), dintg.end(), hout_i);
```

Versión del fragmento en SYCL nativo:

```

1  integrator_rk4* dintg = sycl::malloc_device<integrator_rk4>(
    num_seeds, q_ct1);
2
3  seed_generator gen{static_cast<unsigned>(num_seeds)};
4
5  q_ct1.submit([&](sycl::handler& h){
6      h.parallel_for(sycl::range<1>(num_seeds), [=](sycl::id<1> idx) {
7          const unsigned i = static_cast<unsigned>(idx[0]);
8          dintg[i] = gen(i);
9      });
10 }.wait();
11
12 q_ct1.memcpy(houtput.data() + (0) * num_seeds, dintg, num_seeds *
    sizeof(integrator_rk4)).wait();

```

Usando solamente este fragmento, es posible afirmar que la sintaxis entre CUDA y HIP no es muy diferente. Esto explica también por qué HIPify fue mucho más efectivo durante su migración automatizada, requiriendo cambios mínimos a la base de código.

En adición, en este caso en particular, donde se usa una tarjeta gráfica de marca AMD para el desarrollo, es requerido una configuración adicional para poder ejecutar el código usando la GPU con el compilador de oneAPI⁶ (icpx), y durante una prueba preliminar, presentó

Esto presentó inconvenientes durante la migración y creó la necesidad de incluir a oneAPI como un estándar de portabilidad más a comparar en vez de tomarlo como un sabor diferente de SYCL.

Cabe destacar que ninguna de estas herramientas de migración pudieron satisfactoriamente migrar algunas llamadas de funciones exclusivas de CUDA, en particular *Tex3D.get()*.

⁶Como se indica en las notas de la nueva versión del compilador de oneAPI (Intel Technologies, 2026)

Esta función se encarga de realizar la interpolación trilineal de una textura, que es la función más matemáticamente demandante del algoritmo. A raíz de esto, se modificó el código original para que no se use la configuración y función exclusiva de CUDA. No obstante, esta acomodación es un punto cualitativo de comparación a favor de CUDA.

7 Estado Actual Frente al de Referencia

7.1 Conociendo el estado actual

Con la notación de las arquitecturas objetivo establecidas, al igual que el desarrollo del módulo encargado de la construcción y validación de los archivos de configuración; lo siguiente era la definición del proceso de la comparación entre la arquitectura actual y la arquitectura de referencia. Este proceso, permitirá evaluar el estado del sistema y, por consiguiente, establecer las acciones a tomar con el fin de adaptar la arquitectura hacia el estado objetivo establecido.

Esto requiere conocer el estado actual del sistema, al igual que el conocer el estado objetivo. Siendo así, y ya teniendo la posibilidad de declarar las necesidades de la aplicación, lo siguiente es establecer una manera de determinar el estado del sistema. Dado el enfoque hacia los datos recolectados, es necesario el precisar la manera en la que conoceríamos en qué estado se encuentra el sistema.

Lo primero era el identificar los puntos de acceso por los cuales podríamos acceder a los datos. En el caso de Smart Campus UIS, y teniendo en cuenta que el modelo que se definió depende de los mensajes enviados por los dispositivos, como se observa en la figura 2, existen dos opciones para procesar estos mensajes.

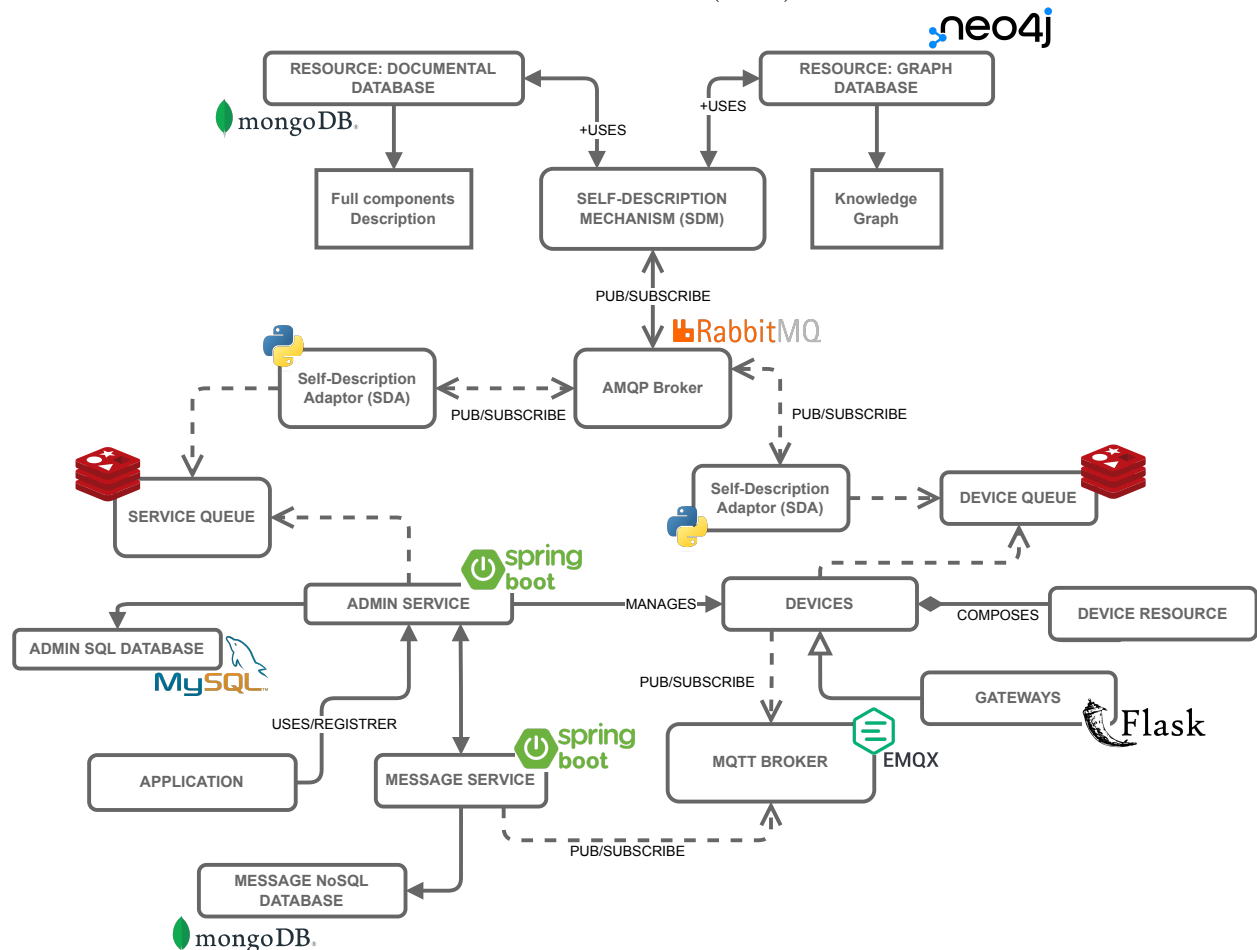
La primera, es consultar los mensajes enviados usando uno de los endpoints del servicio `data_microservice`, lo que da acceso a todos los mensajes registrados. Y, la segunda, usando el mismo bus de datos, empleado por los servicios de la plataforma, lo que permitiría el acceso, incluso de manera más directa, a los mensajes a medida que estos son enviados.

Ahora, cada una de las maneras de acceder a los datos es viable, y podría permitir la

Figura 2:

Arquitectura del prototipo de Smart Campus definido por

Jiménez Herrera (2022)



implementación correspondiente para determinar el estado del sistema. Sin embargo, de entre las dos opciones, se escogió el procesamiento de los mensajes enviados por los dispositivos, enviados por MQTT.

Esto se debe a varios factores, principalmente, debido a cómo se consultan los mensajes desde el endpoint. Para realizar esta consulta, es necesario conocer el *UUID* del dispositivo a consultar lo que, como se evidenció durante la validación de los requerimientos de datos en la sección ??, no es posible.

Aunque sería posible el realizar una modificación al servicio para poder ver todos los mensajes de los dispositivos, esto también podría presentar problemas por la manera en la

que estos se listan. Ya que los mensajes se muestran todos en simultaneo, y no poseen una manera diferenciar 2 mensajes, más allá de su contenido (que no es necesariamente único); tendríamos que guardar todos los mensajes ya procesados y descartarlos cada vez que se consulte el endpoint, lo que generaría un *overhead* el uso de los recursos tanto de memoria como de procesamiento.

Siendo así, queda como opción el broker MQTT para la consulta de los mensajes. Este acercamiento permite el procesamiento, en tiempo real, de los mensajes enviados por los dispositivos, sin la necesidad de conocerlos; al igual que un aprovechamiento de los recursos ya que, una vez procesados, podemos librar el espacio usado. Esto le da una característica al proyecto en forma de una especie de *plug-in*, puesto que, al no requerir modificaciones sobre la plataforma, esta puede funcionar sin ninguna afectación en su estado actual.

Partiendo de lo anterior, se realizó el diseño e implementación de un servicio encargado de procesar los mensajes que están siendo enviados por, y a, cada uno de los dispositivos registrados en Smart Campus UIS.

7.2 Centralizando los Datos

Lo primero para la implementación del observador, era determinar como este tendría acceso al estado de referencia. Hasta el momento, únicamente *Lexical* tiene el contexto definido por el usuario, por lo que era necesario definir una manera mediante la cual otros servicios puedan acceder a este.

Partiendo de la división de responsabilidades, se estableció el implementar un agregador, que contenga el estado definido por el usuario; desde el cual otros servicios puedan acceder a estos datos.

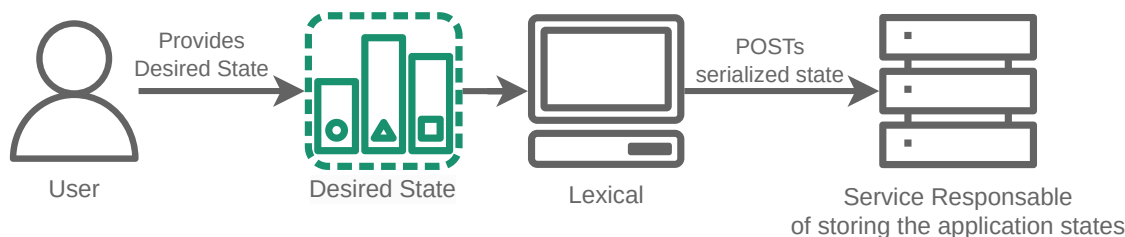
Esto tiene dos ramificaciones adicionales. La primera, es que abre a la posibilidad de declarar más de una aplicación en simultaneo, usando el agregador como un *hub* que permita guardar y consultar los estados de las aplicaciones que se registren, teniendo una instancia de observador por cada una de ellas.

La segunda, posibilita el usar este agregador para almacenar los estados actuales reportados por el observador. Esto se debe a que, las condiciones para evaluar el estado de las aplicaciones, se encuentran en las propiedades definidas por el usuario durante la declaración del estado de referencia. Siendo así, es posible ver el estado objetivo a partir de las propiedades; y el estado actual basado en la evaluación de los componentes registrados en los requerimientos de datos.

De lo anterior, podemos planear un diagrama, visto en la figura 3, que refleje el estado actual de la arquitectura del proyecto.

Figura 3:

Arquitectura actual del proyecto con el agregador



La implementación de este agregador, apodado *Bran*⁷, fue relativamente directa. Este tiene dos requerimientos, el primero, es la capacidad de recibir, guardar y actualizar los estados de las aplicaciones; y segundo, el poder enviar los estados almacenados cada vez que estos se soliciten.

Ambas de estas funcionalidades se implementaron usando el framework *Axum*⁸. Usando como base la librería *StarDuck*, se definió un único endpoint que toma, `apps/:app_name`, que acepta peticiones de tipo GET, para consultar el estado de la aplicación; POST y PUT, para el registro inicial de la aplicación, y posterior actualización de esta.

Ya con el servicio implementado se agregó, a *Lexical*, un cliente HTTP con el cual, tras validar la arquitectura objetivo proveída por el usuario, y como se ve en la figura 3, este realice una petición de tipo POST a la una instancia de *Bran* indicada con el fin de registrarla. Una

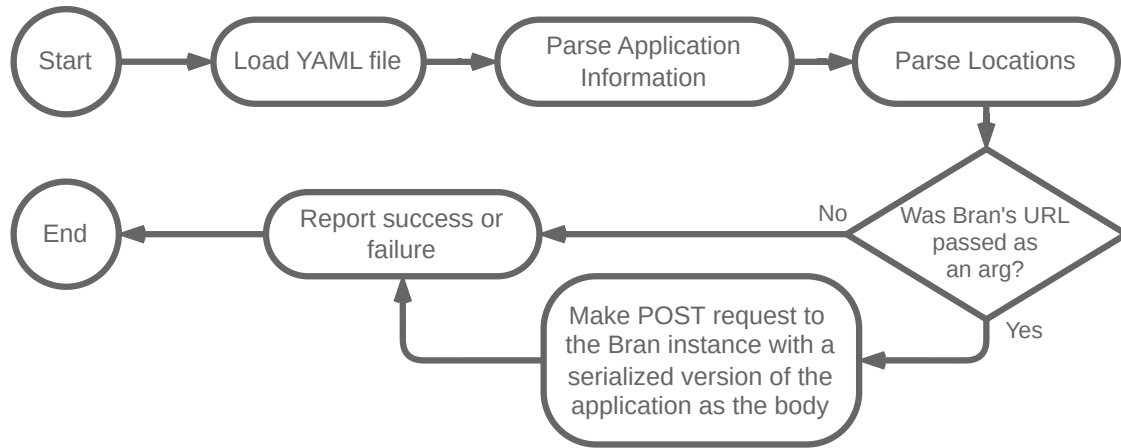
⁷El código fuente de Bran puede encontrarse en el repositorio <https://github.com/ChipDepot/Bran>

⁸Axum es un framework modular para aplicativos web Open Source desarrollado en Rust. Más información sobre este puede encontrarse en su repositorio <https://github.com/tokio-rs/axum>

versión actualizada del proceso realizado *Lexical* puede verse en la figura 4.

Figura 4:

Diagrama de flujo del proceso realizado por Lexical actualizado



7.3 Implementado un Event-Handler

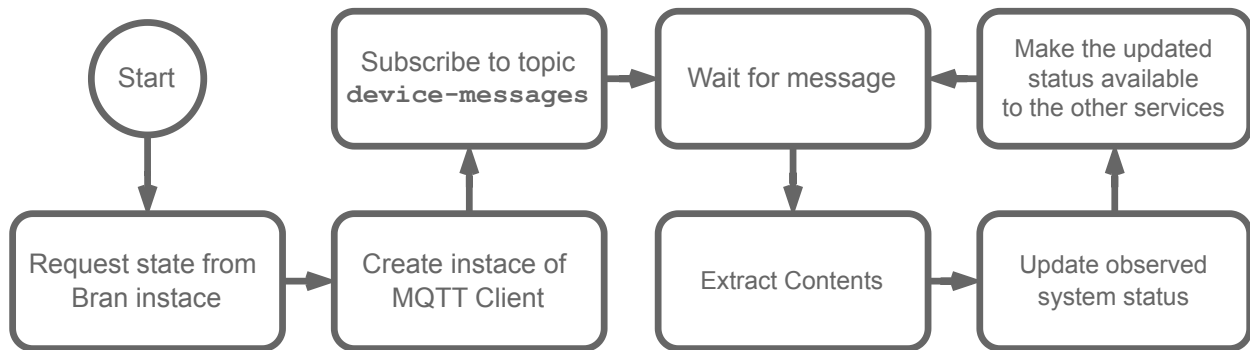
En el contexto de Smart Campus UIS, cada uno de los mensajes enviados por los dispositivos es manejado como un evento. Es decir, cada mensaje recibido debe ser registrado y procesado por los microservicios de administración para lograr un objetivo final. Este procesamiento, recuerda al patrón de diseño *Observer* en el cual se busca el realizar una acción cada vez que el estado, de lo que se está observando, cambia (Shvets, 2019).

Siendo así, se estableció el realizar la implementación de un observador el cual se hará responsable de la captura de los mensajes enviados por los dispositivos y establecer el estado del sistema. De esto, como se puede ver en la figura 5 se propuso un proceso que debía realizar el observador, apodado *Looker*, a implementar.

Este primer acercamiento, es bastante directo en cuanto a la implementación a realizar. Tras adquirir la definición del estado objetivo, se inicializa un cliente de MQTT, conectado el broker usado por Smart Campus UIS y suscrito al tópico usado por los dispositivos, `device-messages` (Jiménez Herrera, 2023), este empezaría a procesar los mensajes a medida que estos van llegando.

Figura 5:

Primera propuesta del proceso a realizar por *Looker*



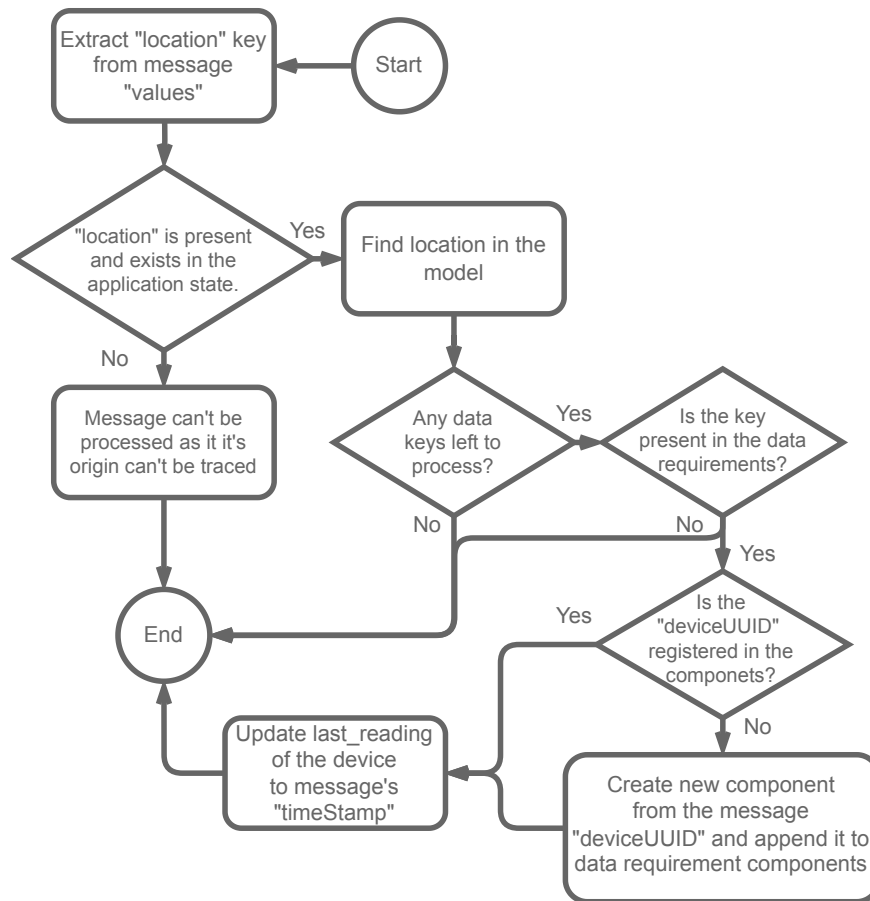
Cada mensaje recibido se somete a un conjunto de pasos para actualizar el estado de la aplicación. La figura 6 describe el proceso que el observador debe realizar con el fin de procesar los mensajes recibidos, dando como resultado, una versión actualizada del estado de la aplicación.

Lo primero a realizar, es ubicarse en el ubicación origen del mensaje. Esto se realiza buscando en las locaciones registradas en la aplicación, usando la llave (Ver figura ??) presente en el mensaje enviado por el dispositivo. En caso que esta, no esté presente, o la locación no haya sido referenciada en la declaración del estado objetivo, el mensaje será descartado. Se debe resaltar que, este puede ser uno de los principales puntos de falla debido a los posibles errores que se puedan presentar al usar nombres diferentes para una locación, sea en los mensajes enviados por problemas en la configuración; o en el momento de la creación del estado de referencia.

Una vez localizado el origen geográfico del mensaje, se recorrerán los datos reportados por los dispositivos. Estos se cruzarán contra los requerimientos de datos, actualizando los tiempos de los componentes; o registrándolos en caso de que sea el primer mensaje publicado por ellos. Esto se realizará para todas los datos enviados por el dispositivo.

Figura 6:

Proceso realizado por el observador durante el consumo de los mensajes enviados por los dispositivos



7.4 Comparando Arquitecturas

Ahora que se conoce el estado actual del sistema, al igual que estado de referencia; lo siguiente a realizar era la comparación de las arquitecturas. El resultado de esta, daría la base para poder decidir el qué hacer para adaptar el sistema y el cómo hacerlo.

Es necesario definir el como se realizará la evaluación del sistema. Esto no es tan sencillo como hacer una comparación usando un igual ($A == B$), ya que, el realizar esto, no resultaría realmente información sobre, qué, internamente, presenta problemas.

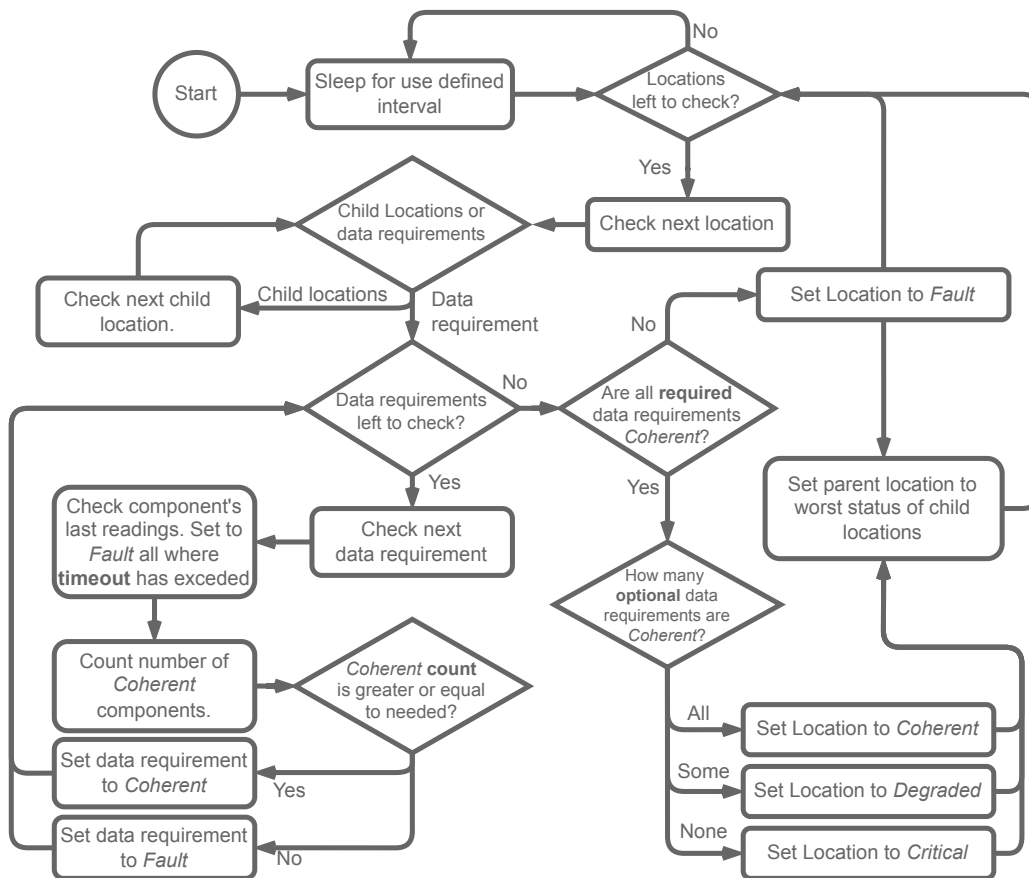
Ahora, la propiedades implementadas, definidas en los estados objetivo; son la manera las características principales por las cuales es posible realizar la evaluación del sistema. *Timeout*,

cumpliendo la función de establecer el tiempo máximo entre los reportes de dispositivos, definiendo cuando estos pueden considerarse como "vencidos"; y *count*, como la cantidad de dispositivos en estado *Coherent* mínima para el desarrollo de la aplicación.

Siendo así, se definió un proceso de comparación consta de evaluar cada componente del sistema en función de las propiedades. A partir de estas, es posible el evaluar el estado de la aplicación. Este proceso es descrito por la figura 7.

Figura 7:

Proceso reloj realizado por el observador para la actualización del estado de la aplicación



El proceso, apodado reloj, debido a su ejecución dado un intervalo de tiempo definido por el usuario; realiza la evaluación de la aplicación, recorriendo la estructura locación por locación. Siendo así, y con el fin de evitar una dispersión grande de los lugares de procesamiento, se implementó en *Looker*.

Los componentes, pueden existir únicamente en un estado binario, sea *Coherent*, en el caso

de que cumplan con las condiciones establecidas en el requerimiento; o *Fault* de lo contrario. Estos se basan únicamente en la propiedad *timeout*, marcándolos de manera correspondiente a partir de su último reporte registrado.

Así mismo, los requerimientos de datos, con la misma característica de su estado binario; usan propiedad *count* para definir su condición. En caso de que la cantidad total de componentes sea igual a la requerida, se marcará como *Coherent*; de lo contrario, estará en *Fault*. Se ha de resaltar que, debido a que estas propiedades son opcionales durante la declaración, podrán presentarse casos en los que, dada una configuración específica, con un único reporte (o incluso ninguno) estos podrán considerarse válidos durante todo el de ejecución de la aplicación.

Una vez evaluadas los componentes y los requerimientos de datos, lo siguiente es evaluar la locación. Esto dependerá del tipo de la locación. Si es un agrupador, su estado será el peor de los estados de las hijas. De lo contrario, si tiene requerimientos de datos, su estado depende de sus requerimientos de datos y del tipo de cada uno de ellos.

El estado de estas locaciones se determina en dos pasos. En el primer paso, se verifica si alguno de sus requerimientos obligatorios está en estado *Fault*. En ese caso, la locación se considera en estado *Fault*, ya que no puede cumplir con los requisitos mínimos de la aplicación. Para las locaciones sin requerimientos opcionales, este será el final del proceso.

El segundo paso, en el caso de locaciones con requerimientos opcionales, el factor determinante será la cantidad de estados *Coherent*. La locación tendrá un estado *Critical*, *Degraded*, o *Coherent* según si todos, algunos o ninguno de estos requerimientos está en estado *Fault*, respectivamente.

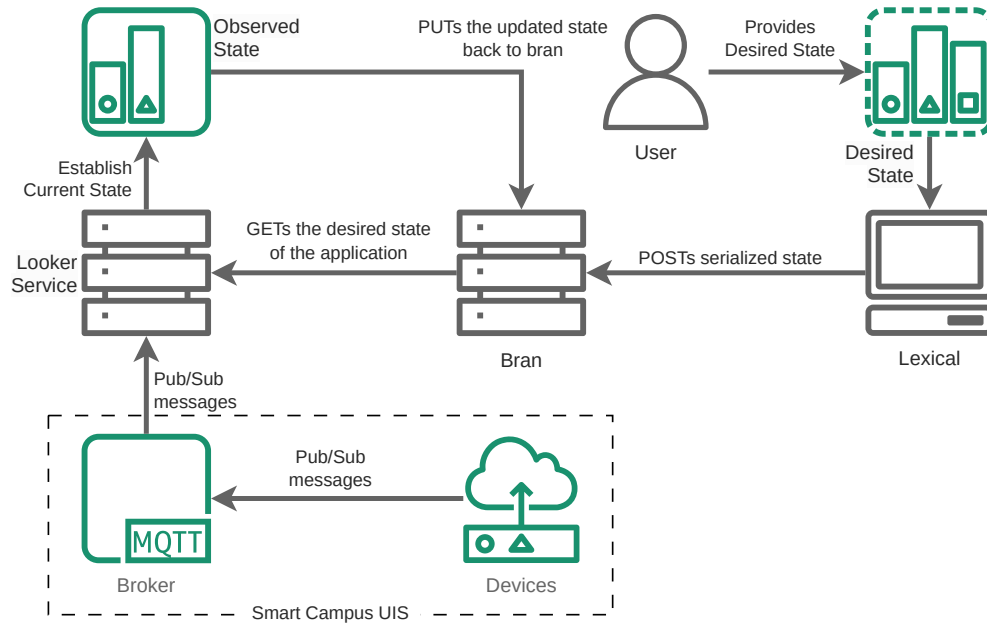
Esta manera de evaluar las locaciones permite el tener una idea más clara del estado de cada una de las locaciones, y por consiguiente, el estado de la aplicación. A partir de esta evaluación, se realizarán las diferentes adaptaciones necesarias con el fin de acercar el estado de la aplicación al estado de referencia esperado.

Una vez actualizado, y evaluado el estado; se tendría que reportar al agregador el nuevo

estado de la aplicación. Para esto, se agregó un paso final tras la ejecución del proceso reloj, en la cual, *Looker*, realiza una petición tipo PUT al endpoint designado en *Bran*, con el fin de actualizar el registro. La arquitectura del proyecto, hasta el momento, se puede ver en la figura 8.

Figura 8:

Arquitectura actual del proyecto



8 Adaptando la Arquitectura

8.1 Identificando Los Problemas, Estableciendo Acciones

Ya con una manera de evaluar los estados observados de las aplicaciones definidas, lo siguiente a realizar era el establecer un proceso por el cual se pudieran identificar los problemas en la aplicación, y a partir de esto, establecer las acciones a realizar con el fin de modificar la arquitectura de la aplicación hacia el estado de referencia definido.

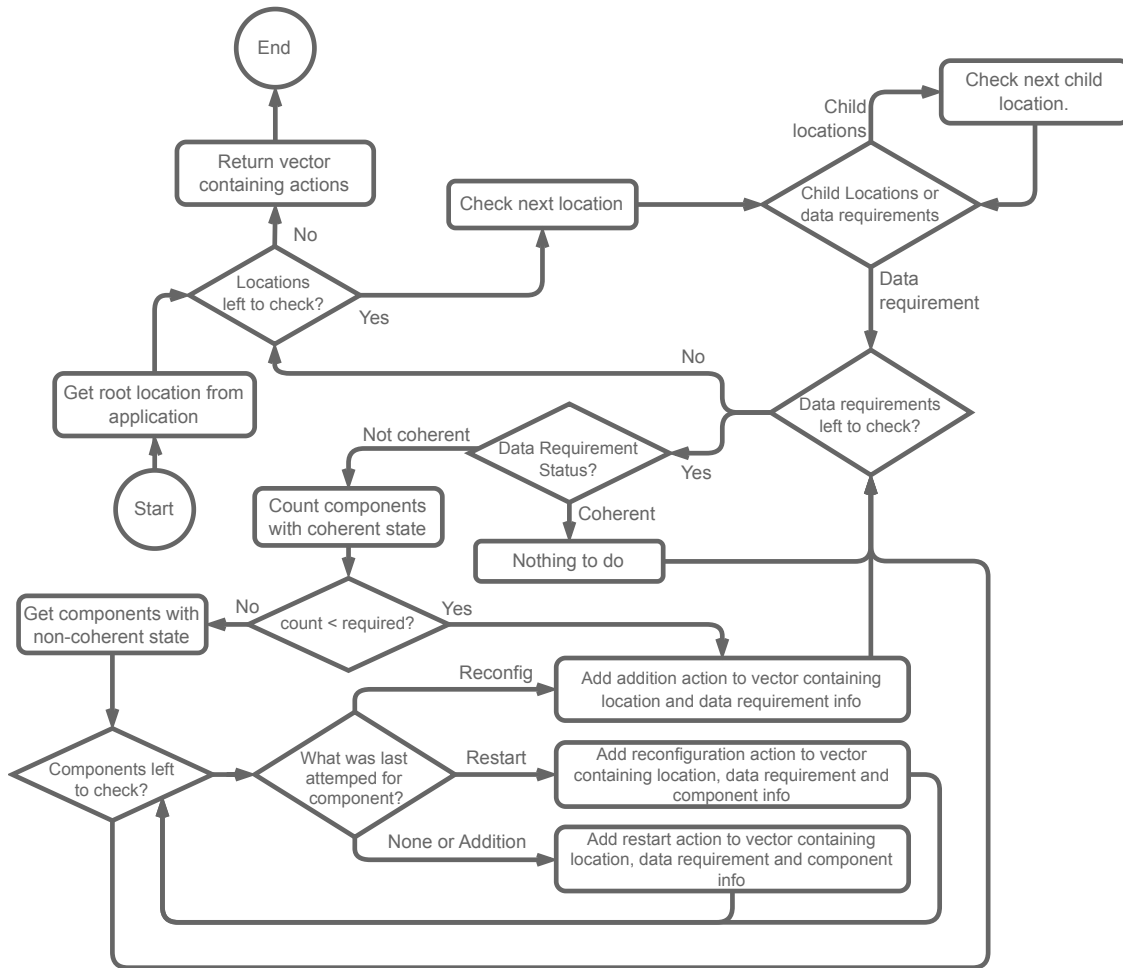
Como se estableció durante la sección 7.4, el estado de la aplicación, en términos generales, es determinada por los estados de sus requerimientos de datos. Siendo así, se definió un proceso por el cual, a partir de las condiciones en las que se encuentren estos requerimientos,

se determinen las acciones a realizar con el fin de adaptar la arquitectura.

Partiendo de esto, se definió el proceso, descrito por la figura 9, en dos partes. La primera, está encargada de la búsqueda e identificación de los problemas dentro de la aplicación; y la segunda, de manera interna, el determinar las acciones a seguir para adaptar la arquitectura.

Figura 9:

Proceso para la identificación de problemas de los requerimientos de datos



La manera en la de que se identifican los problemas es similar a la vista en la figura 7. Se recorren las locaciones, buscando requerimientos de datos. Una vez en una de estas locaciones, se validan los requerimientos y se definen las acciones.

Ahora, las posibles acciones a tomar para la arquitectura de la aplicación, como se vió durante la sección 3.1.3, dependen de los objetivos y los requerimientos del sistema. Siendo

así, se tomaron 3 tipos de acciones por las cuales se puede modificar el estado del sistema: *Addition*, *Restart* y *Reconfigure*.

Todos los requerimientos en un estado diferente a *Coherent*, son evaluados con el fin de establecer la acción a realizar. Lo primero, es revisar la cantidad de componentes registrados en el requerimiento; si se tiene menos de los requeridos, se realizará un acción tipo *Addition* con el fin de cumplir con el numero de fuentes de datos esperada.

En el caso de tener la cantidad requerida de dispositivos, pero estado aún en estado *Fault*; la siguiente opción es reiniciar el servicio. Esto busca poner nuevamente en un estado válido el componente, en caso de que haya quedado en un estado inválido; o poner nuevamente en ejecución el servicio en el escenario de que este haya muerto.

Si el reiniciar no tiene efecto en el estado del componente, la siguiente opción es reconfigurar el componente. Esto busca la modificación de algún aspecto, sea la actualización una variable de ambiente o parámetro, con el fin de retornarlo a un estado válido.

Finalmente, si todo falla y no es posible recuperar el dispositivo; la última opción es iniciar un nuevo servicio el cual pueda suplir las condiciones de su predecesor. De esta manera, aún es posible retornar la aplicación hacia un estado válido, independiente de los componentes en un estado irrecuperable.

La implementación de este proceso, se realizó en el agregador *Bran*. Esto debido a que, al tener el estado de las aplicaciones, cortesía de lo observado por *Looker*; deja a este servicio como el punto medio de la aplicación, encargado de definir las acciones a realizar.

8.2 De Acciones, a Instrucciones

Ahora, se tienen las acciones a realizar para poder adaptar la arquitectura de la aplicación hacia el estado de referencia; sin embargo, estas acciones no contienen la información necesaria para ejecutar ninguno de los procesos necesarios.

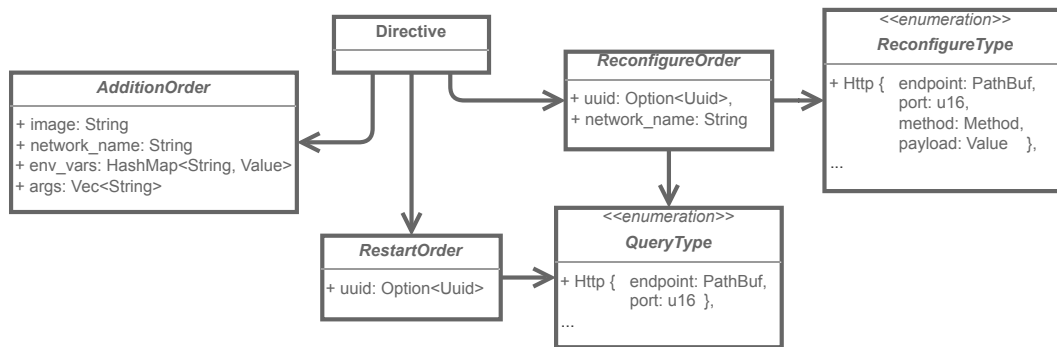
Partiendo de esto, se determinó la necesidad de una colección estructuras de datos, que contuvieran las instrucciones a seguir, para cada una de las acciones definidas. Esta, debe

especificar la locación de la aplicación en la cual se ejecutan, con el fin de tener una mejor granularidad y control sobre la manera en la que se realizan las modificaciones en la arquitectura.

En consecuencia, como se observa en la figura 10, se definió una estructura de datos llamada *Directive*. Esta contiene tres campos adicionales para las estructuras *AdditionOrder*, *RestartOrder*, y *ReconfigureOrder*; que corresponden con las acciones *Addition*, *Restart* y *Reconfigure*, respectivamente.

Figura 10:

UML de la estructura de directivas y ordenes



Cada una estas directivas, y ordenes, se agregan a un registro en *Bran*, usando el nombre de la aplicación y la locación a la pertenecen para su indexación. Cada una se registra usando un endpoint dedicado a la recepción de las órdenes.

La orden de adición contiene el nombre de la imagen del servicio a desplegar, la red a la cual debe conectarse para poder interactuar con los demás servicios, y los argumentos que se deben pasar sea como variables de ambiente o argumentos de línea de comando.

Las órdenes de reinicio y reconfiguración poseen el *UUID* del dispositivo que buscan afectar. Siendo así, al ser necesario tener una manera por la cual identificar qué servicio es qué componente, se implementó el enum *QueryType*. Este, contiene la información de como debe buscarse el contenedor en la red. Para esta primera implementación, su única variante es una búsqueda *Http* hacia un endpoint y un puerto.

Finalmente, la orden de reconfiguración tiene como parte extra la red en la que se bus-

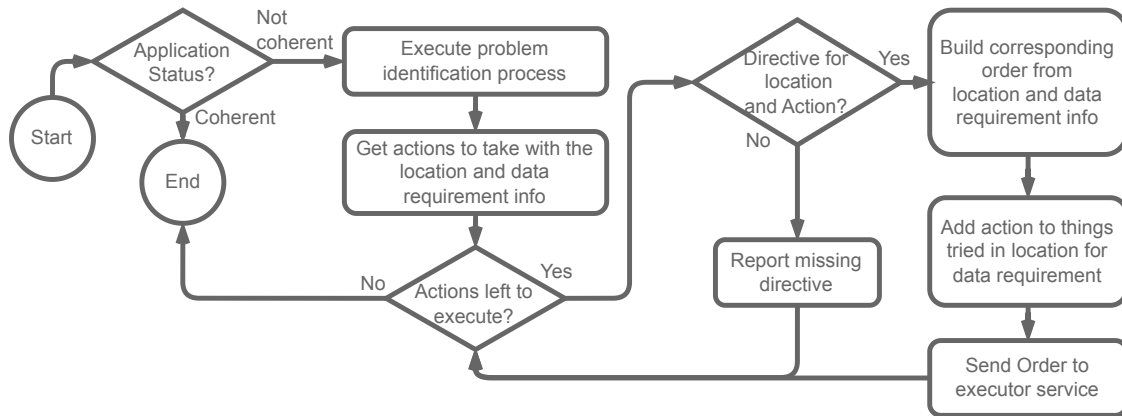
para el servicio a reiniciar y el tipo de reconfiguración (*ReconfigureType*) a realizar. Este funciona de manera similar a *QueryType*, en tanto indica la manera en la que se aplicará la reconfiguración. En este caso, para la variante *Http* implementada, indica el endpoint y el puerto a usar, junto con el método y payload que deben ir en la petición para hacer la reconfiguración efectiva.

Se ha de resaltar que, estas ordenes, aunque contienen las instrucciones para poder realizar modificaciones a la arquitectura, en el momento de su declaración, no están completas y requieren de información extra para poder ejecutarse.

El proceso de la figura 11, muestra el proceso por el cual las órdenes presentes en el registro de directivas de *Bran*, son usadas como base para la construcción de las órdenes finales, y son enviadas para su posterior ejecución.

Figura 11:

Proceso de decisión de acciones base realizado por *Bran*



Su ejecución inicia validando el estado de la aplicación reportada. En caso de este sea diferente a *Coherent*, se llamará al proceso de identificación de problemas para obtener las acciones a realizar para alterar el estado de aplicación.

Una vez recibido el vector conteniendo las acciones, y la información asociada correspondiente, se realizará la construcción de cada una de las órdenes a ejecutar. Esta construcción se realiza a partir de las instrucciones bases establecidas en las directivas usando el UUID para las acciones de *Restart* y *Reconfigure*; y la información de la locación y el requerimiento

de dato en las acciones de *Addition*⁹.

Ya con las ordenes creadas, estas se enviaran al servicio encargado de ejecutar las instrucciones, dando como resultado, modificaciones en la arquitectura.

8.3 De Instrucciones a Acciones

Ya con las instrucciones necesarias para poder realizar cambios en la arquitectura de las aplicaciones, se realizó la implementación de un servicio encargado de ejecutar las ordenes recibidas desde *Bran*.

Ahora, Smart Campus UIS, como plataforma, se ejecuta sobre contenedores de docker; y para efectos de esta primera versión del proyecto, toda la arquitectura de presente, se ejecuta sobre contenedores. Como consecuencia, era necesario que el servicio implementado, tuviera la de ejecutar comandos de docker con el fin de poder llevar a cabo las acciones.

Esto se solucionó usando la Docker Engine API como el canal de comunicación entre el servicio, y el resto de la arquitectura. Esto se debe a que, esta API REST, permite la interacción directa con el daemon de Docker (Docker Inc, 2023), posibilitando la ejecución de todos los comandos relacionados con docker, desde el iniciar y detener servicios hasta crear y eliminar contenedores.

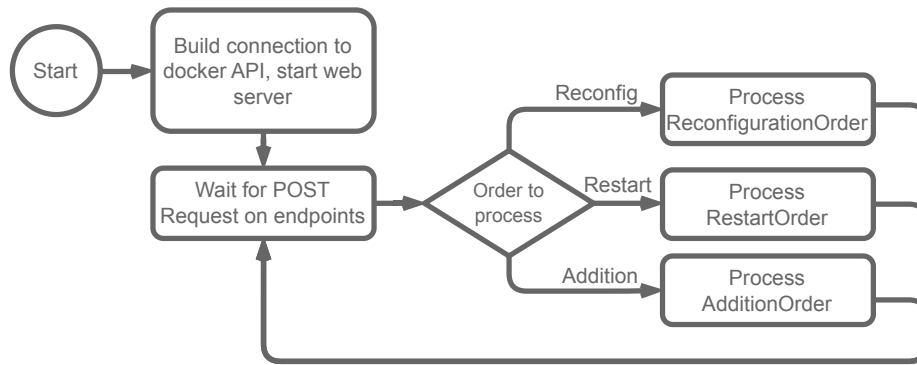
El servicio, apodado *DoThing*, realiza visto en la figura 12. Este está orientado principalmente al procesamiento de las peticiones realizadas, por lo que es relativamente sencillo en cuanto a las acciones que este realiza.

Tras inicializar la conexión con la API de Docker, iniciará un servidor web por el cual estará recibiendo las órdenes a ejecutar. Cada tipo de acción tiene su propio endpoint donde se realiza un proceso específico para poder ejecutarla. Cada acción realizada, registrará los contenedores afectados en un registro interno, con el fin de facilitar el procesamiento de múltiples acciones sobre un servicio.

⁹Es necesario resaltar que estas construcciones, en especial para el caso de las acciones de *Addition*, fueron implementadas para funcionar con los servicios *Mocker* desarrollados específicamente para el proyecto. Esto se explorará más a fondo durante la sección 9 y 11.

Figura 12:

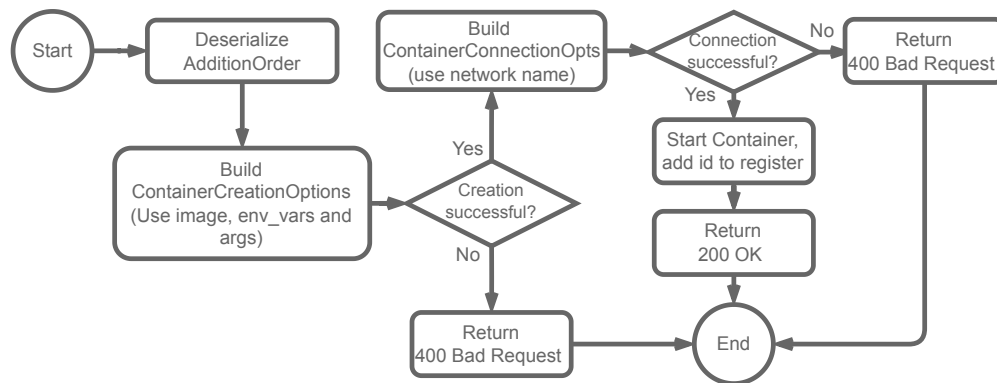
Proceso base de DoThing



Las ordenes de *Addition*, como se referencia en la figura 13, realizan el proceso a partir de la *image*, *env_vars* y *args*, declarados en la orden, para construir el contenedor con su respectivo servicio.

Figura 13:

Proceso para la ejecución de ordenes de adición



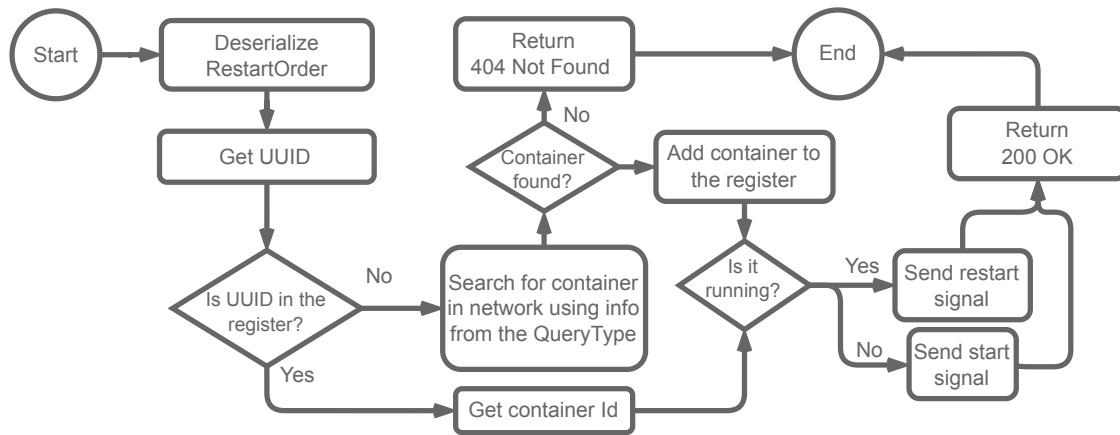
Una vez se tenga el contenedor creado, usando **network_name**, se conectará el contenedor creado a la red indicada. De no presentarse errores durante este proceso, se iniciará el contenedor, se agrega al registro y retorna 200 OK.

Las órdenes de tipo *Restart*, referenciadas en la figura 14 pueden ejecutarse de una de dos maneras, dependiendo del estado del servicio a afectar.

Primero, se realiza la búsqueda del contenedor en el registro; de no estar presente, se realizará la búsqueda de este usando la información proveída en el *QueryType* de la orden.

Figura 14:

Proceso para la ejecución de ordenes de reinicio

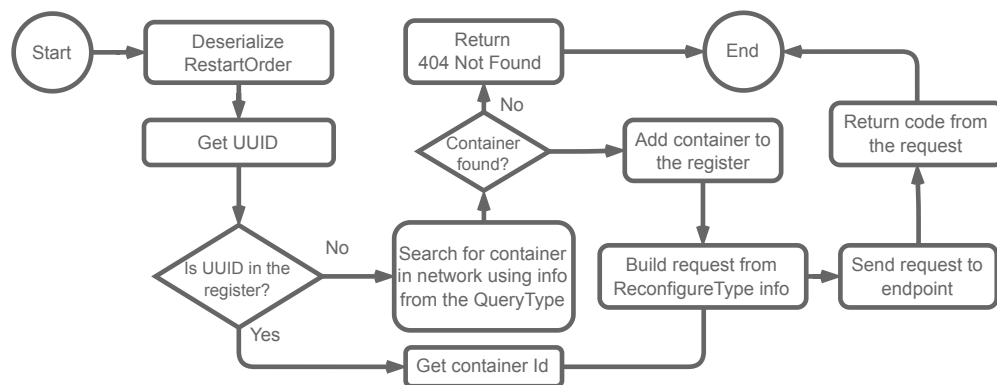


Una vez identificado el servicio, si está corriendo¹⁰, la orden de reinicio será equivalente al comando `docker restart <container>`, esto con el objetivo de eliminar *cuelgues* en el servicio. Por el contrario, si el contenedor no está en ejecución, la orden ejecutará el equivalente al comando `docker start <container>`, para así iniciar nuevamente el servicio.

Finalmente, las órdenes de tipo *Reconfigure*, cuyo proceso puede verse en la figura 15, es similar al realizado para las órdenes de tipo *Restart*.

Figura 15:

Proceso para la ejecución de ordenes de reconfiguración



Una vez ubicado el servicio, se construirá, a partir de los datos reportados en *ReconfigureType*, la petición a realizar en el servicio. Esta, en consecuencia, se enviará usando un

¹⁰Referido en la documentación de Docker Engine API como *Running*

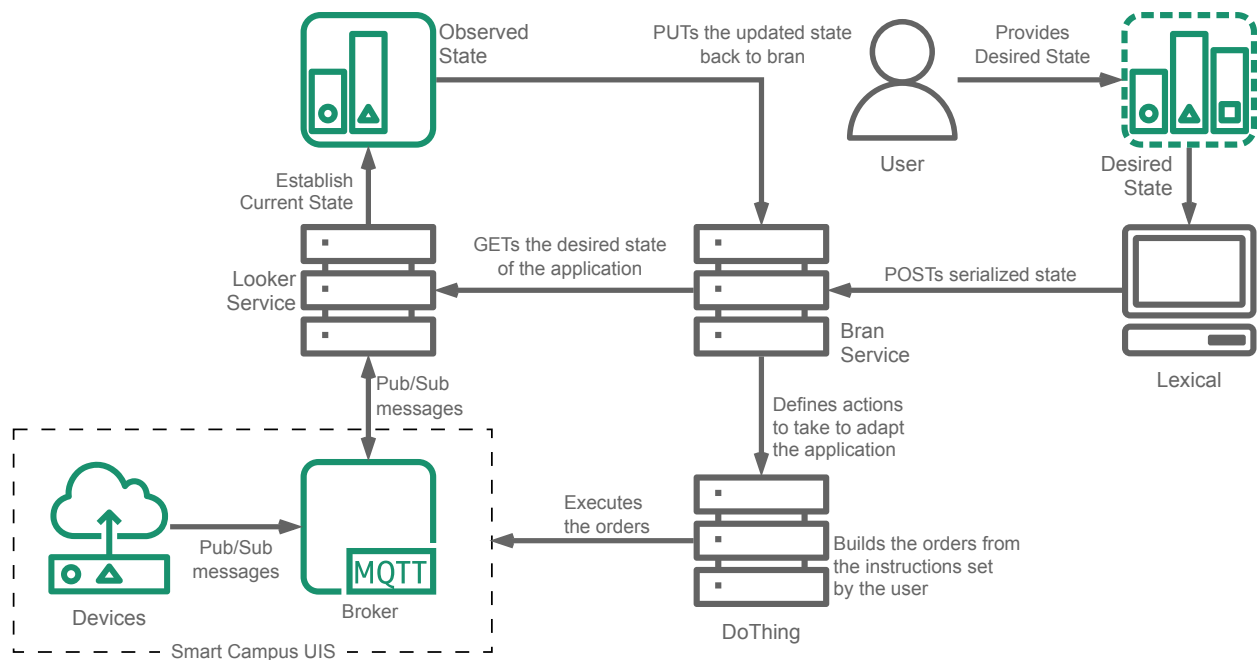
cliente *Http* y esperará a la respuesta de esta. El resultado, de esta transacción, será el mismo reportado por *DoThing* hacia *Bran*.

Ahora, con la implementación de este servicio, encargado de ejecutar las acciones requeridas para la adaptación de las arquitecturas de las aplicaciones, completa el ciclo autónomo MAPE-K implementado para Smart Campus UIS.

Una vez ejecutadas las órdenes; el efecto de estas, serían captadas por el observador, evaluando nuevamente el estado de la aplicación, e, idealmente, reportando un cambio positivo hacia la coherencia esperada con el estado de referencia definido en un principio.

La arquitectura final del presente proyecto, con toda la implementación de los servicios, puede ver se en la figura 16.

Figura 16:
Arquitectura completa del proyecto



9 Validación de la Implementación

Lo último a realizar fue la validación de las funcionalidades de la aplicación desarrollada. Para ello, se establecieron escenarios de prueba por los cuales se pueda comprobar el funcio-

namiento de aplicación. Estos, se definieron con el fin de probar la capacidad y la efectividad de los mecanismos implementados de modificar el estado inicial de la aplicación hacia el estado de referencia, al igual que la identificación de algunas de las falencias de esta.

Ahora, se debe resaltar que, las pruebas se realizaron usando un servicio genérico, apodado Mocker, encargado de la generación de datos, especialmente desarrollado para funcionar en condiciones ideales.

Para cada una de estas pruebas, se definió un estado de referencia, dadas unas condiciones iniciales, y una serie de eventos los cuales modificarán nuevamente el estado de la aplicación, las cuales tendrán que ser nuevamente manejadas.

9.1 Escenario Experimental A

El primer escenario, busca el evaluar la capacidad de la implementación realizada de iniciar toda una aplicación desde cero. Partiendo de esto, se definió la aplicación vista en la figura 17. Esta tiene dos locaciones raíz, con requerimientos de datos iguales en ambas.

Figura 17:

Diagrama del escenario experimental A

Location - North Sector	Location - South Sector
Required: + 1 temperature component reporting every 30 seconds + 3 wind-speed component reporting every 60 seconds	Required: + 1 temperature component reporting every 30 seconds + 3 wind-speed component reporting every 60 seconds
Optional: + 3 Particulate Matter components reporting every 120 seconds	Optional: + 3 Particulate Matter components reporting every 120 seconds

Inicialmente, no habrán dispositivos presentes en ejecución, por lo que el sistema tendrá que identificar la falla, definir las acciones de adición correspondientes, y ejecutarlas para poder suplir con los requerimientos de datos.

Para ello, tomando como referencia la figura 17, se realizó la declaración del estado de

referencia, presente en el anexo 1, y, como observa en la figura 18, se validó usando *Lexical*.

Figura 18:

Validación de la declaración de referencia realizada para el escenario experimental A

```
INFO lexical] Lexical started
INFO lexical::parsing::parser] Loading file
INFO lexical::parsing::parser] YAML File loaded
INFO lexical::parsing::parser] Creating Application instance
INFO lexical::parsing::parser] Application instance created
INFO lexical::parsing::parser] Creating Locations
INFO lexical::location::location] Processing temperature in Sector Norte
INFO lexical::location::location] Processing wind-speed in Sector Norte
INFO lexical::location::location] Processing particulate-matter in Sector Norte
INFO lexical::location::location] Processing wind-speed in Sector Sur
INFO lexical::location::location] Processing particulate-matter in Sector Sur
INFO lexical::location::location] Processing temperature in Sector Sur
INFO lexical::parsing::parser] Locations created
INFO lexical] Application Malaga definition was validated!
```

Ya con las condiciones iniciales, y el estado de referencia definido, se estructuraron, las directivas, vistas en el anexo 2, que permitirían realizar la adaptación de la arquitectura hacia el estado de objetivo.

Partiendo de todo lo anterior, se definió la serie de pasos a realizar para este primer escenario experimental:

1. Desplegar los servicios de Smart Campus UIS
2. Desplegar los servicios *Bran* y *DoThing*.
3. Usando *Lexical*, establecer el estado de referencia de la aplicación.
4. Declarar en los endpoints de *Bran*, las directivas definidas.
5. Desplegar el servicio *Looker* para la aplicación.
6. A la nivelación del estado de la aplicación.

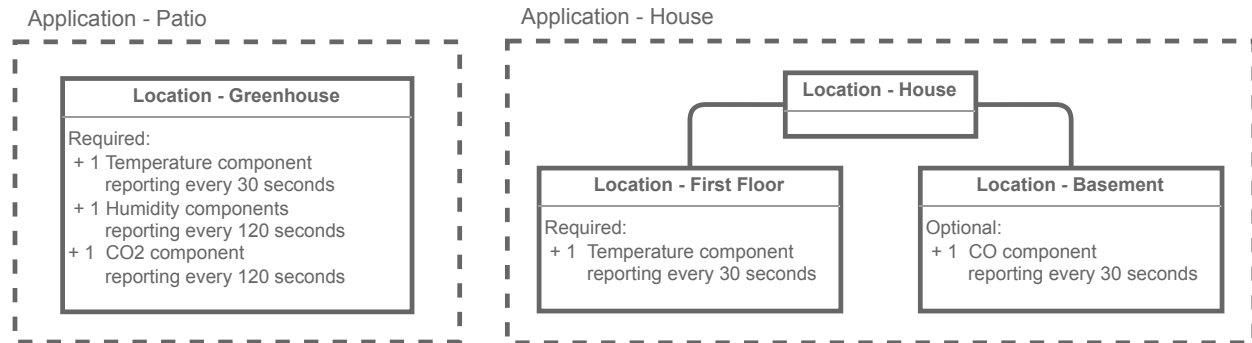
Al final de la simulación de este escenario, se eliminarán todos los servicios desplegados con el fin de asegurar una ejecución limpia de futuros procesos.

9.2 Escenario Experimental B

El segundo escenario, busca el evaluar la capacidad de la implementación en donde ya existen componentes en ejecución, al igual que el manejo de múltiples aplicaciones. Para esto, se definieron dos aplicaciones, como se observa en la figura 19,

Figura 19:

Aplicaciones definidas para el desarrollo del escenario experimental B



Para esta simulación, todos los componentes de la aplicación ya estarán ejecutándose, con el problema de que no todos estarán dentro de los parámetros establecidos. Esto requerirá que la aplicación identifique el servicio correspondiente, y realice la reconfiguración de sus parámetros.

De la misma manera que se realizó en escenario experimental A, se realizó la declaración de los estados de referencia de cada una de las aplicaciones, al igual que las directivas para su ejecución. Los YAML de esta declaración, y las directivas definidas, se encuentran en los apéndices 3, 4, 5 y 6.

Partiendo de todo lo anterior, se definió la serie de pasos a realizar para este primer escenario experimental:

1. Desplegar los servicios de Smart Campus UIS
2. Desplegar los servicios *Bran* y *DoThing*.
3. Usando *Lexical*, establecer el estado de referencia de la aplicación Patio y House.
4. Declarar en los endpoints de *Bran*, las directivas definidas.
5. Desplegar los servicios de simulación de datos *Mocker* respecto a las expectativas, tal que el estado sea coherente.
6. Desplegar los servicios *Looker* para cada una de las aplicaciones.

7. Matar los servicios desplegados y eliminar uno de los contenedores creados.
8. Esperar a ejecución de las órdenes *Restart* y *Addition*.

10 Análisis de Resultados

10.1 Resultados De Escenario Experimental A

El objetivo de las pruebas realizadas en el presente escenario experimental, era validar que, los procesos implementados, efectivamente realizaran la adaptación de las aplicaciones desde un estado de completa falla, en donde ninguno de los datos necesarios estuviera presente.

Siguiendo los pasos definidos en la sección 9.1, se realizó el despliegue de los servicios base de Smart Campus UIS, al igual que *Bran* y *DoThing*. Como se presenta en la figura 21, se registró el estado de referencia al agregador tras validarlo usando *Lexical*, y se fijaron las directivas a usar para la adaptación de la arquitectura.

Figura 20:

Aplicación y directivas registradas en Bran

```

$ docker run -d -p 8014:8014 --name=bran --network=smart_campus_production_smart_uis -e RUST_LOG=bran mrdal
$ docker logs -f bran
d3d9ede81dbbb8a372bb79f2868dcb8f959b3af5331b23bb36f0c6701cb92ad5
[2024-01-19T06:38:53Z INFO bran::planner::planner] Starting Planner Execution
[2024-01-19T06:38:53Z INFO bran] Initializing server at 0.0.0.0:8014
[2024-01-19T06:39:28Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53004
[2024-01-19T06:39:28Z INFO bran::endpoints::receptor] Malaga was added to the register
[2024-01-19T06:39:34Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53024
[2024-01-19T06:39:34Z INFO bran::endpoints::receptor] Added addition directive in north-sector in app Malaga
[2024-01-19T06:39:38Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53024
[2024-01-19T06:39:38Z INFO bran::endpoints::receptor] Added addition directive in south-sector in app Malaga
[2024-01-19T06:39:40Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53024
[2024-01-19T06:39:40Z INFO bran::endpoints::receptor] Updated restart directive in north-sector in app Malaga
[2024-01-19T06:39:45Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53024
[2024-01-19T06:39:45Z INFO bran::endpoints::receptor] Updated restart directive in south-sector in app Malaga
[2024-01-19T06:39:54Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53024
[2024-01-19T06:39:54Z INFO bran::endpoints::receptor] Updated reconfig directive in north-sector in app Malaga
[2024-01-19T06:39:57Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53024
[2024-01-19T06:39:57Z INFO bran::endpoints::receptor] Updated reconfig directive in south-sector in app Malaga

```

Una vez se inició el servicio *Looker*, como se ve en la figura 21, este empezó a evaluar el estado de la aplicación; que al no tener ningún tipo de reporte de algún componente, entró directamente en estado de falla.

El estado observado, se reportó a *Bran*, el cual, realizó el recorrido para identificar los puntos de falla de la aplicación. Como se observa en la figura 22, como era de esperar, los

Figura 21:

Looker evalúa el estado de la aplicación

```
9aa81c397805164b178c8fece1a19c5dac97485fa49
ERROR looker::eyes::clock] Failed to fetch timeout check value: environment variable not found
INFO looker::eyes::mqtt] Using URL tcp://mqtt_broker:1883/
WARN looker::eyes::clock] Defaulting timeout_check to 10
INFO looker] Starting server at 0.0.0.0:3000
INFO looker::eyes::mqtt] Establishing connection...
INFO looker::eyes::mqtt] Created MQTT connection
INFO looker::eyes::clock] Timeout check at 2024-01-19 01:46:53.704225448
INFO looker::eyes::clock] Timeout check complete
INFO looker::axe::requester] Updated application Malaga has been POSTed to http://bran:8014/apps/Malaga
INFO looker::eyes::clock] Timeout check at 2024-01-19 01:47:03.705439088
INFO looker::eyes::clock] Timeout check complete
INFO looker::axe::requester] Updated application Malaga has been POSTed to http://bran:8014/apps/Malaga
```

tres requerimientos definidos en cada una de las locaciones estaba en estado de falla. Al no haber componentes reportando datos, como era de esperarse, se definieron acciones *Addition* para todos los componentes de la aplicación.

Figura 22:

Bran identifica los problemas y establece las acciones

```
bran::endpoints::receptor] Malaga's state was updated
bran::planner::planner] Starting Planner Execution
bran::planner::planner] Checking Application Malaga
bran::planner::planner] Found 3 data requirements with errors in south-sector
bran::planner::planner] Creating Addition Order for data requirement particulate-matter in south-sector
bran::planner::planner] Creating Addition Order for data requirement wind-speed in south-sector
bran::planner::planner] Creating Addition Order for data requirement temperature in south-sector
bran::planner::planner] Found 3 data requirements with errors in north-sector
bran::planner::planner] Creating Addition Order for data requirement temperature in north-sector
bran::planner::planner] Creating Addition Order for data requirement particulate-matter in north-sector
bran::planner::planner] Creating Addition Order for data requirement wind-speed in north-sector
bran::planner::planner] Executing Addition order
bran::planner::planner] Building addition order 1 out of 2 from ProblemInfo { location_key: "south-sector",
  se_uuid: None }
bran::planner::planner] Executing order 1 out of 2
```

A partir de las acciones, se crearon las órdenes para ejecución usando las directivas como base. Como se puede ver en la figura 23, cada una de estas se envió a *DoThing*, el cual realizó el procesamiento requerido para la construcción y arranque de los contenedores. El resultado de este procesamiento se puede ver en la figura 24

En total, se crearon 12 contenedores, que corresponden a los 12 componentes requeridos para el desarrollo de la aplicación. Cada uno de ellos configurado para suplir las necesidades de operación establecidas en el estado de referencia. Estos despliegues, como era de esperarse, fueron detectado por *looker*, el cual realizó el nuevas valoraciones del estado.

Los valores por defecto de los servicios usados para estas pruebas es un intervalo de tiempo entre mensajes de un minuto. Este valor por defecto, aunque para los requerimientos de datos

Figura 23:

DoThing ejecuta las acciones establecidas por Bran

```

0 dothing::endpoints::director] Using ContainerCreateOpts { name: None, params: {"Env": Array [String("ip=mqtt_broker",
device_uuid=ae33cddf-7f70-407f-878c-71d198dc9934")], "Image": String("mrdahaniel/mocker:latest"), "Cmd": Array [String("topic:wind-speed")], "Topic": String("topic:wind-speed")}] } as creation options
0 dothing::endpoints::director] Sending create signal to docker socket
0 dothing::endpoints::director] Container created with name: pedantic-elgamal
0 dothing::endpoints::director] Building network connection options...
0 dothing::endpoints::director] Built network connection options
0 dothing::endpoints::director] Connecting container to network smart_campus_production_smart_uis
0 dothing::endpoints::director] Connected container to network smart_campus_production_smart_uis
0 dothing::endpoints::director] Starting container pedantic-elgamal
0 dothing::endpoints::director] Started container pedantic-elgamal
0 dothing::endpoints::director] Process successful!
0 dothing::endpoints::director] POST request from 172.19.0.6:54336 to add container
0 dothing::endpoints::director] Building container...
0 dothing::actions::addition] Processing env_vars from Hash to Vec
0 dothing::actions::addition] env_vars processed
0 dothing::actions::addition] Building ContainerCreationOptions...
0 dothing::actions::addition] Built ContainerCreationOptions
0 dothing::endpoints::director] Using ContainerCreateOpts { name: None, params: {"Image": String("mrdahaniel/mocker:latest"), "Cmd": Array [String("topic:wind-speed")], "Topic": String("topic:wind-speed")}] } as creation options
0 dothing::endpoints::director] Sending create signal to docker socket

```

cuyos *timeout*, sean superiores, supliría las necesidades de la aplicación. Sin embargo, en el marco de esta simulación, esta frecuencia de reportes no es suficiente para cumplir con el estado de referencia de los requerimiento de temperatura de las locaciones.

Como consecuencia, el estado de la aplicación regresó a estado *Fault*, y *Bran*, al detectar los problemas, emitió las acciones tipo *Restart* con el fin de traer los contenedores a un estado válido. Esto se puede ver en la figura 25.

Figura 25:

Bran identifica otros problemas en la aplicación

```

0 planner::planner] Starting Planner Execution
0 planner::planner] Checking Application Malaga
0 planner::planner] Found 1 data requirements with errors in south-sector
0 planner::planner] Creating Restart Order for component d8ef9ba1-1a9a-4e08-a0e6-e2414
0 planner::planner] Found 1 data requirements with errors in north-sector
0 planner::planner] Creating Restart Order for component cc81a342-41f4-4301-9c14-33287
0 planner::planner] Executing Restart order
0 planner::planner] Executing order: RestartOrder { uuid: Some(d8ef9ba1-1a9a-4e08-a0e6-e2414) }
0 endpoints::receptor] PUT for Malaga request from 172.19.0.8:53374
0 endpoints::receptor] Malaga's state was updated
0 planner::planner] Executing Restart order
0 planner::planner] Executing order: RestartOrder { uuid: Some(cc81a342-41f4-4301-9c14-33287) }

```

Este intento de reiniciar los servicios, como era de esperarse, no fue realmente efectiva debido a que la falla se debe a un problema en la configuración del servicio. Siendo así, la

Figura 24:

Contenedores creados por DoThing en respuesta a las órdenes recibidas

```
Every 1.0s: docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
be10fac91e31	mrdaaniel/mocker:latest	"/mocke wind-speed..."	7 seconds ago	Up 6 seconds
87be21eb8a92	mrdaaniel/mocker:latest	"/mocke wind-speed..."	7 seconds ago	Up 6 seconds
1493f42f41c0	mrdaaniel/mocker:latest	"/mocke wind-speed..."	8 seconds ago	Up 7 seconds
elbe9dc6ce92	mrdaaniel/mocker:latest	"/mocke particulat..."	8 seconds ago	Up 7 seconds
2fe1387443f9	mrdaaniel/mocker:latest	"/mocke particulat..."	8 seconds ago	Up 8 seconds
6f33df9571d7	mrdaaniel/mocker:latest	"/mocke temperatur..."	9 seconds ago	Up 8 seconds
8ac5e9d279a1	mrdaaniel/mocker:latest	"/mocke temperatur..."	9 seconds ago	Up 8 seconds
249e342bfce9	mrdaaniel/mocker:latest	"/mocke wind-speed..."	10 seconds ago	Up 9 seconds
190d3fe013c4	mrdaaniel/mocker:latest	"/mocke wind-speed..."	10 seconds ago	Up 9 seconds
447a4ed21267	mrdaaniel/mocker:latest	"/mocke wind-speed..."	10 seconds ago	Up 10 seconds
765944fa10fd	mrdaaniel/mocker:latest	"/mocke particulat..."	11 seconds ago	Up 10 seconds
7d6cf546e876	mrdaaniel/mocker:latest	"/mocke particulat..."	11 seconds ago	Up 10 seconds
cd1718c40388	mrdaaniel/looker:latest	"/looker"	2 minutes ago	Up 2 minutes
9a4eb8c81017	mrdaaniel/dotthing:latest	"/dotthing"	10 minutes ago	Up 10 minutes
d3d9ede81dbb	mrdaaniel/bran:latest	"/bran -e watcher_i..."	10 minutes ago	Up 10 minutes
80f4928c8d58	smart-campus_production-data_microservice	"java -jar /app.jar"	6 days ago	Up 12 hours

aplicación se mantuvo en estado de falla. Se esperaba que, ambos componentes encargados de la temperatura, al ya haber descartado el reinicio como un mecanismo efectivo para esta situación, fueran sometidos a una acción de reconfiguración, sin embargo, como se observa en la figura 26, en ese instante, sólo el componente de **north-sector** fue afectado.

Esta reconfiguración fue ejecutada por *DoThing*, y tomó efecto, alterando el valor por defecto entre mensajes, pasando de un reporte cada minuto, a uno cada 25 segundos¹¹. Con el nuevo valor, los componentes de temperatura, empezaban a reportar con mayor frecuencia, cumpliendo con las expectativas.

Poco tiempo después, el componente de temperatura de **south-sector**, reportó el estado de falla, por lo se ejecutó la misma acción de reconfiguración que con el componente de la otra locación, llevando el estado de la aplicación a *Coherent*.

A partir de esta simulación, se pudo validar que el proceso implementado, tiene la capa-

¹¹Según lo definido en las directivas presentes en el apéndice 2

Figura 26:

Bran establece acciones de reconfiguración para adaptar la aplicación

```

receptor] PUT for Malaga request from 172.19.0.8:50694
receptor] Malaga's state was updated
receptor] PUT for Malaga request from 172.19.0.8:49084
receptor] Malaga's state was updated
anner] Starting Planner Execution
anner] Checking Application Malaga
anner] Found 1 data requirements with errors in north-sector
anner] Creating Reconfigure Order for component cc81a342-41f4-4301-9c14-33
anner] Found 0 data requirements with errors in south-sector
anner] Executing Reconfigure order
anner] Executing order: ReconfigureOrder { uuid: Some(cc81a342-41f4-4301-9
endpoint: "/get/device", port: 8000 }, reconfig: Http { endpoint: "/updat
Number(25)} } }
receptor] PUT for Malaga request from 172.19.0.8:55762
receptor] Malaga's state was updated

```

ciudad de realizar la adaptación desde el peor escenario, hacia un esto de coherencia con la aplicación declarada inicialmente. Todas las acciones definidas por *Bran*, permitieron modificar el estado de manera positiva.

Sin embargo, se identificaron algunas limitaciones de esta implementación. Como se observa en la figura 26, se muestra que **south-sector** no presenta problemas en sus requerimientos, cuando, debido a su configuración, debía estar en estado de falla.

Esto se debe a un *quirk* de la implementación realizada. Al manejar los procesos cada cierto intervalo de tiempo, es posible que un componente se encuentren en estado de falla, pero alcance a pasar a coherente justo antes de la evaluación de las acciones a tomar. Eventualmente, el componente fallaría en reportar antes de una una de las evaluaciones desencadenando en una acción, como fue el caso con la tardía reconfiguración del componente de **south-sector**.

10.2 Resultados De Escenario Experimental B

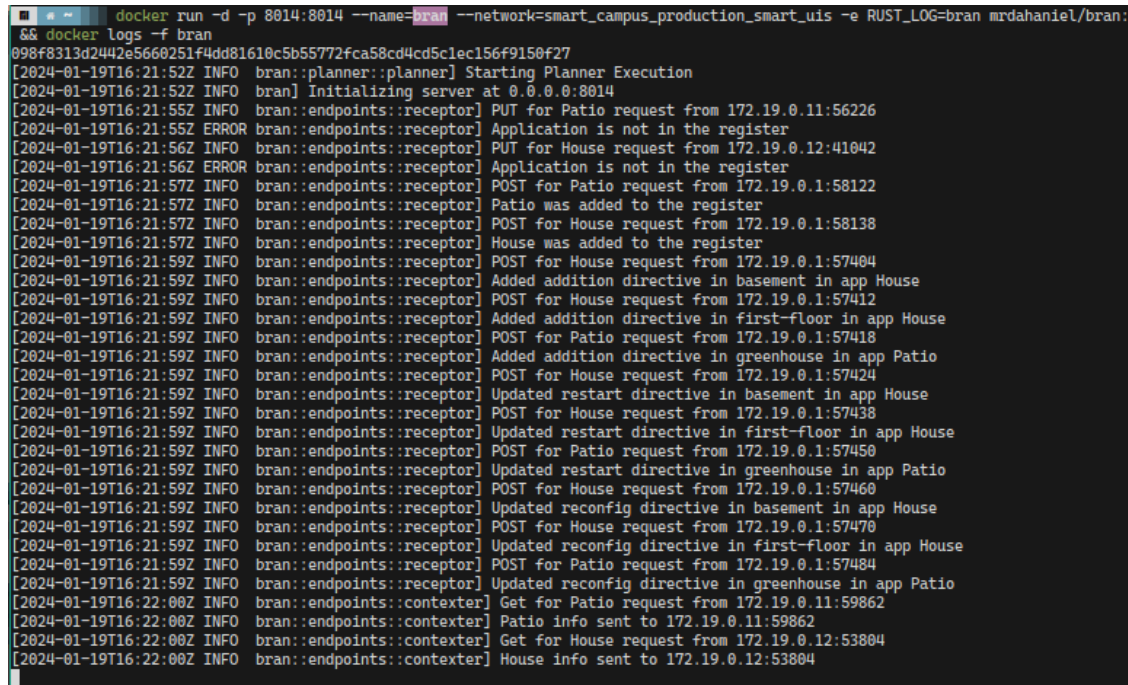
El objetivo principal del escenario experimental B, era evaluar la capacidad de la implementación realizada de recuperar una aplicación en estado de coherencia, en donde se presenta una falla masiva, nuevamente a un estado válido; al igual que la capacidad de Bran

de manejar más de una aplicación, como se había nombrado en la sección 7.2.

Igual que durante el escenario experimental A, se realizó el despliegue de los servicios *Bran* y *DoThing*. Como se observa en la figura 27, se definieron las arquitecturas objetivo de las dos aplicaciones a monitorear, al igual que las directivas a usar durante este proceso.

Figura 27:

Aplicaciones y directivas registradas en Bran



```

$ docker run -d -p 8014:8014 --name=bran --network=smart_campus_production_smart_uis -e RUST_LOG=bran mrdahaniel/bran:
$ docker logs -f bran
098f8313d2442e5660251f4dd81610c5b55772fca58cd4cd5c1ec156f9150f27
[2024-01-19T16:21:52Z INFO bran::planner::planner] Starting Planner Execution
[2024-01-19T16:21:52Z INFO bran] Initializing server at 0.0.0.0:8014
[2024-01-19T16:21:55Z INFO bran::endpoints::receptor] PUT for Patio request from 172.19.0.11:56226
[2024-01-19T16:21:55Z ERROR bran::endpoints::receptor] Application is not in the register
[2024-01-19T16:21:56Z INFO bran::endpoints::receptor] PUT for House request from 172.19.0.12:41042
[2024-01-19T16:21:56Z ERROR bran::endpoints::receptor] Application is not in the register
[2024-01-19T16:21:57Z INFO bran::endpoints::receptor] POST for Patio request from 172.19.0.1:58122
[2024-01-19T16:21:57Z INFO bran::endpoints::receptor] Patio was added to the register
[2024-01-19T16:21:57Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:58138
[2024-01-19T16:21:57Z INFO bran::endpoints::receptor] House was added to the register
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:57404
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Added addition directive in basement in app House
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:57412
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Added addition directive in first-floor in app House
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for Patio request from 172.19.0.1:57418
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Added addition directive in greenhouse in app Patio
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:57424
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Updated restart directive in basement in app House
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:57438
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Updated restart directive in first-floor in app House
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for Patio request from 172.19.0.1:57450
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Updated restart directive in greenhouse in app Patio
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:57460
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Updated reconfig directive in basement in app House
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:57470
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Updated reconfig directive in first-floor in app House
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for Patio request from 172.19.0.1:57484
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Updated reconfig directive in greenhouse in app Patio
[2024-01-19T16:22:00Z INFO bran::endpoints::contexter] Get for Patio request from 172.19.0.11:59862
[2024-01-19T16:22:00Z INFO bran::endpoints::contexter] Patio info sent to 172.19.0.11:59862
[2024-01-19T16:22:00Z INFO bran::endpoints::contexter] Get for House request from 172.19.0.12:53804
[2024-01-19T16:22:00Z INFO bran::endpoints::contexter] House info sent to 172.19.0.12:53804

```

Seguidamente, como se observa en la figura 28, se desplegaron los servicios de *Looker* para cada una de las aplicaciones a monitorear. Ya con esta base lista, y como se definió en los pasos a seguir, se desplegaron de manera manual, un total de 3 servicios *Mocker* con configuraciones específicas (tanto de los datos que estos reportaban, como de los tiempos de reporte) con el fin de cumplir con los requerimientos de datos definidos para las dos aplicaciones.

Figura 28:

Dos instancias de *looker* monitoreando las aplicaciones declaradas

```

looker::eyes::clock] Defaulting timeout_check to 10
looker::eyes::mqtt] Using URL tcp://mqtt_broker:1883/
looker] Starting server at 0.0.0.0:3000
looker::eyes::mqtt] Establishing connection...
looker::eyes::mqtt] Created MQTT connection
looker::app::application] message received from topic: 'device-messages'
looker::app::application] device_uuid = dact68bb-18ad-4ef2-82b8-4abee5742bd2
looker::app::application] topic = temperatura
looker::app::application] values = {"location":"entrance","temperature":28}
looker::app::application] No location was found for key 'entrance'
looker::app::application] message received from topic: 'device-messages'
looker::app::application] device_uuid = 7b5185e3-6181-452e-a647-fb6957a76425
looker::app::application] topic = temperatura
looker::app::application] values = {"co2":44,"location":"basement"}
looker::app::application] No location was found for key 'basement'
looker::app::application] message received from topic: 'device-messages'
looker::app::application] device_uuid = 15f66f9f-bda5-4e4b-844c-fe687bb12979
looker::app::application] topic = temperatura
looker::app::application] values = {"co2":185,"humidity":50,"location":"gree
looker::app::application] Updated state.
looker::eyes::clock] Timeout check at 2024-01-19 11:22:10.325554880
looker::eyes::clock] Timeout check complete
looker::axe::requester] Updated application Patio has been POSTed to http://
looker::eyes::clock] Timeout check at 2024-01-19 11:22:20.326572796
looker::eyes::clock] Timeout check complete
looker::axe::requester] Updated application Patio has been POSTed to http://

[2024-01-19T16:22:00Z INFO looker::eyes::mqtt] Using URL tcp://mqtt_broker:1883/
[2024-01-19T16:22:00Z WARN looker::eyes::clock] Defaulting timeout_check to 10
[2024-01-19T16:22:00Z INFO looker] Starting server at 0.0.0.0:3000
[2024-01-19T16:22:00Z INFO looker::eyes::mqtt] Establishing connection...
[2024-01-19T16:22:00Z INFO looker::eyes::mqtt] Created MQTT connection
[2024-01-19T16:22:01Z INFO looker::app::application] message received from topic:
[2024-01-19T16:22:01Z INFO looker::app::application] device_uuid = dact68bb-18ad-4
[2024-01-19T16:22:01Z INFO looker::app::application] topic = temperatura
[2024-01-19T16:22:01Z INFO looker::app::application] values = {"location":"entranc
[2024-01-19T16:22:01Z ERROR looker::app::application] No location was found for key
[2024-01-19T16:22:01Z INFO looker::app::application] message received from topic:
[2024-01-19T16:22:01Z INFO looker::app::application] device_uuid = 7b5185e3-6181-4
[2024-01-19T16:22:01Z INFO looker::app::application] topic = temperatura
[2024-01-19T16:22:01Z INFO looker::app::application] values = {"co2":44,"location":
[2024-01-19T16:22:01Z INFO looker::app::application] Updated state.
[2024-01-19T16:22:02Z INFO looker::app::application] message received from topic:
[2024-01-19T16:22:02Z INFO looker::app::application] device_uuid = 15f66f9f-bda5-4
[2024-01-19T16:22:02Z INFO looker::app::application] topic = temperatura
[2024-01-19T16:22:02Z INFO looker::app::application] values = {"co2":185,"humidity
[2024-01-19T16:22:02Z ERROR looker::app::application] No location was found for key
[2024-01-19T16:22:10Z INFO looker::eyes::clock] Timeout check at 2024-01-19 11:22:
[2024-01-19T16:22:10Z INFO looker::eyes::clock] Timeout check complete
[2024-01-19T16:22:10Z INFO looker::axe::requester] Updated application House has b
[2024-01-19T16:22:20Z INFO looker::eyes::clock] Timeout check at 2024-01-19 11:22:
[2024-01-19T16:22:20Z INFO looker::eyes::clock] Timeout check complete
[2024-01-19T16:22:20Z INFO looker::axe::requester] Updated application House has b

```

Como era de esperarse, con las configuraciones hechas a la medida de las aplicaciones, el estado de estas era *Coherent*, sin la necesidad de ningún tipo de cambios en los dispositivos. Siendo así, como se ve en la figura 29, se matan los 3 contenedores desplegados, y se elimina uno de ellos.

Figura 29:

Se matan los contenedores con los servicios iniciales

```
Every 1.0s: docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
013ed7ae8477	mrdaaniel/mockers:latest	"/mockers co2:random..."	3 minutes ago	Exited (137) 1 second ago
da06a86bc3c0	mrdaaniel/mockers:latest	"/mockers temperatur..."	3 minutes ago	Exited (137) 1 second ago
e5958ed4acac	mrdaaniel/looker:latest	"/looker"	3 minutes ago	Up 3 minutes
01->3000/tcp	mrdaaniel/looker:latest	"/looker"	3 minutes ago	Up 3 minutes
4864c04bc106	mrdaaniel/looker:latest	"/looker"	3 minutes ago	Up 3 minutes
00->3000/tcp	mrdaaniel/looker:latest	"/looker"	3 minutes ago	Up 3 minutes
025c7b091086	mrdaaniel/doing:latest	"/doing"	3 minutes ago	Up 3 minutes
50->8050/tcp	mrdaaniel/doing:latest	"/doing"	3 minutes ago	Up 3 minutes
c70fec2f5626	mrdaaniel/bran:latest	"/bran -e watcher_i..."	4 minutes ago	Up 4 minutes
14->8014/tcp	mrdaaniel/bran:latest	"/bran -e watcher_i..."	4 minutes ago	Up 4 minutes
80f4928c8d58	smart_campus_production-data_microservice	"java -jar /app.jar"	7 days ago	Up 25 hours
91->8091/tcp	smart_campus_production-data_microservice	"java -jar /app.jar"	7 days ago	Up 25 hours
b8888aee8a63	mongo-express	"/sbin/tini -- /dock..."	7 days ago	Up 25 hours
81->8081/tcp	smart_campus_production-mongo-express-1	"/sbin/tini -- /dock..."	7 days ago	Up 25 hours
a8d97ecef21	mongo	"docker-entrypoint.s..."	7 days ago	Up 25 hours
27017->27017/tcp	smart_campus_production-mongo-1	"docker-entrypoint.s..."	7 days ago	Up 25 hours
e3a7ce6eb973	eclipse-mosquitto	"/docker-entrypoint.s..."	7 days ago	Up 25 hours
83->1883/tcp	mqtt_broker	"/docker-entrypoint.s..."	7 days ago	Up 25 hours

La figura 30, muestra el como *Bran*, define las órdenes para reiniciar los servicios para traer la aplicación a un estado nuevamente. En condiciones normales, se esperaría que esto sea lo único a realizar para adaptar la arquitectura, sin embargo, este no fue el caso.

Figura 30:

Bran intenta reiniciar los contenedores

```

r] Starting Planner Execution
r] Checking Application Patio
r] Found 1 data requirements with errors in greenhouse
r] Creating Restart Order for component 0b60df47-b823-4c92-b065-a17696c140f6 data requirement temperature in
r] Executing Restart order
r] Executing order: RestartOrder { uuid: Some(0b60df47-b823-4c92-b065-a17696c140f6), query_type: Http { endp
r] Checking Application House
r] Found 1 data requirements with errors in first-floor
r] Creating Restart Order for component 4f8c7b1b-3682-4734-82ca-33af6909c652 data requirement temperature in
r] Found 1 data requirements with errors in basement
r] Creating Restart Order for component 6e33c6a3-d942-4e0d-8daf-e0f967d74b90 data requirement co in basement
r] Executing Restart order
r] Executing order: RestartOrder { uuid: Some(4f8c7b1b-3682-4734-82ca-33af6909c652), query_type: Http { endp
r] Executing Restart order
r] Executing order: RestartOrder { uuid: Some(6e33c6a3-d942-4e0d-8daf-e0f967d74b90), query_type: Http { endp
r] PUT for Patio request from 172.19.0.8:42000

```

Ya que los contenedores no fueron desplegados, o ha sido modificados, de ninguna manera por *DoThing*, no estos no están mapeados. Siendo así, como se ve en la figura 31, es necesario buscar los contenedores en la red, pero, al estar los servicios muertos, y depender de la consulta *Http* para poder identificarlos, no puede encontrarlos.

Figura 31:

DoThing no puede encontrar los contenedores

```

:restart] Making get request to http://house-looker:8000/get/device
:restart] Got no response from http://house-looker:8000/get/device
:restart] Checking container with id 4864c04bc10660d3ce16384bde5c173cladbb495c1146068fe9c93deaaeb37a2
:restart] Making get request to http://patio-looker:8000/get/device
:restart] Got no response from http://patio-looker:8000/get/device
:restart] Checking container with id 025c7b091086226a66a8251b80451c70b9aec99137ac45802f6a1339217a295b
:restart] Making get request to http://dothing:8000/get/device
:restart] Got no response from http://dothing:8000/get/device
:restart] Checking container with id c70fec2f56261c8e3e572c83173cdcbf708eac0b30f1acd256dae1e7ac23c1b9
:restart] Making get request to http://bran:8000/get/device
:restart] Got no response from http://bran:8000/get/device
:restart] Checking container with id 80f4928c8d5886801e2a91a77d5d1043ca43c8d3d308badfcae10546b441f6bc9
:restart] Making get request to http://data_microservice:8000/get/device
:restart] Got no response from http://data_microservice:8000/get/device
:restart] Checking container with id b8888aee8a63ffbc1f6ffbb09f06cbe09af770de511bb16bf417233eac7b2572
:restart] Making get request to http://smart_campus_production-mongo-express-1:8000/get/device
:restart] Got no response from http://smart_campus_production-mongo-express-1:8000/get/device
:restart] Checking container with id a8d97ecef219fc6e79f009a3a398351224671e4e724c02b2296aaba395a83b6
:restart] Making get request to http://smart_campus_production-mongo-1:8000/get/device
:restart] Got no response from http://smart_campus_production-mongo-1:8000/get/device
:restart] Checking container with id e3a7ce6eb973ebf6c95b955db1787d8aa430335a3d53c1554f967b95025ba51a
:restart] Making get request to http://mqtt_broker:8000/get/device
:restart] Got no response from http://mqtt_broker:8000/get/device
:director] Could not find container

```

Esto demuestra otra de las falencias con la implementación realizada, que es la búsqueda del crecimiento de la fase de conocimiento. El haber identificado los componentes, antes de que se presentaran problemas, hubiera posibilitado la ejecución de órdenes cuando los servi-

cios estén caídos. Así mismo, se hubiera podido establecer una búsqueda que no dependiera del estado de los componentes, quizás usando tags.

El resultado final, es que *Bran*, al no poder actuar de otra manera, como se observa en la figura 32, ordena acciones de adición para suplir los componentes. Esto, eventualmente, resulta en el regreso de la aplicación a un estado válido, pero sigue sin ser lo ideal.

Figura 32:

Bran crea órdenes de adicción

```
[2024-01-19T19:21:38Z INFO bran:endpoints:receptor] PUT for House request from 172.19.0.9:57962
[2024-01-19T19:21:38Z INFO bran:endpoints:receptor] House's state was updated
[2024-01-19T19:21:38Z INFO bran:planner:planner] Executing Addition order
[2024-01-19T19:21:38Z INFO bran:planner:planner] Building addition order 1 out of 1 from ProblemInfo { location_key: "greenhouse", data_requirement_key: "
temperature", device_uuid: None }
[2024-01-19T19:21:38Z INFO bran:planner:planner] Executing order 1 out of 1
[2024-01-19T19:21:38Z INFO bran:planner:planner] Executing Restart order
[2024-01-19T19:21:38Z INFO bran:planner:planner] Executing order: RestartOrder { uuid: Some(a22dacd5-e63b-4052-9266-4445aa19bf37), query_type: Http { endp
oint: "/get/device", port: 8000 } }
[2024-01-19T19:21:47Z INFO bran:endpoints:receptor] PUT for Patio request from 172.19.0.8:43622
[2024-01-19T19:21:47Z INFO bran:endpoints:receptor] Patio's state was updated
[2024-01-19T19:21:48Z INFO bran:endpoints:receptor] PUT for House request from 172.19.0.9:41636
[2024-01-19T19:21:48Z INFO bran:endpoints:receptor] House's state was updated
[2024-01-19T19:21:49Z INFO bran:planner:planner] Executing Addition order
[2024-01-19T19:21:49Z INFO bran:planner:planner] Building addition order 1 out of 1 from ProblemInfo { location_key: "greenhouse", data_requirement_key: "
co2", device_uuid: None }
[2024-01-19T19:21:49Z INFO bran:planner:planner] Executing order 1 out of 1
[2024-01-19T19:21:49Z INFO bran:planner:planner] Checking Application House
[2024-01-19T19:21:49Z WARN bran:planner:planner] Found 1 data requirements with errors in first-floor
[2024-01-19T19:21:49Z INFO bran:planner:planner] Creating Addition Order for data requirement temperature in first-floor
[2024-01-19T19:21:49Z INFO bran:planner:planner] Creating Restart Order for component b05984f0-ad28-46d7-a052-9eb14b7a7a15 data requirement temperature in
first-floor
[2024-01-19T19:21:49Z WARN bran:planner:planner] Found 1 data requirements with errors in basement
[2024-01-19T19:21:49Z INFO bran:planner:planner] Creating Addition Order for data requirement co in basement
[2024-01-19T19:21:49Z INFO bran:planner:planner] Creating Restart Order for component 198a2e59-b52f-4be5-b02c-30b05a28130b data requirement co in basement
[2024-01-19T19:21:49Z INFO bran:planner:planner] Executing Addition order
[2024-01-19T19:21:49Z INFO bran:planner:planner] Building addition order 1 out of 1 from ProblemInfo { location_key: "first-floor", data_requirement_key:
"temperature", device_uuid: None }
[2024-01-19T19:21:49Z INFO bran:planner:planner] Executing order 1 out of 1
[2024-01-19T19:21:50Z INFO bran:planner:planner] Executing Restart order
[2024-01-19T19:21:50Z INFO bran:planner:planner] Executing order: RestartOrder { uuid: Some(b05984f0-ad28-46d7-a052-9eb14b7a7a15), query_type: Http { endp
oint: "/get/device", port: 8000 } }
[2024-01-19T19:21:57Z INFO bran:endpoints:receptor] PUT for Patio request from 172.19.0.8:50608
[2024-01-19T19:21:57Z INFO bran:endpoints:receptor] Patio's state was updated
[2024-01-19T19:21:58Z INFO bran:endpoints:receptor] PUT for House request from 172.19.0.9:51078
[2024-01-19T19:21:58Z INFO bran:endpoints:receptor] House's state was updated
[2024-01-19T19:22:01Z INFO bran:planner:planner] Executing Addition order
[2024-01-19T19:22:01Z INFO bran:planner:planner] Building addition order 1 out of 1 from ProblemInfo { location_key: "basement", data_requirement_key: "co
", device_uuid: None }
[2024-01-19T19:22:01Z INFO bran:planner:planner] Executing order 1 out of 1
```

11 Conclusiones y Trabajo Futuro

El presente trabajo buscaba realizar la implementación de mecanismos de adaptación de arquitecturas a la plataforma Smart Campus UIS, que permitiera alterar el estado de una aplicación hacia un estado de referencia, de manera autonómica.

Para ello, se desarrolló e implementó un metamodelo capaz representar dichas arquitecturas, al igual que una manera de declararla tomando como foco los datos requeridos para su correcto funcionamiento; el cual cumple con el objetivo de establecer un estado de referencia.

También se diseñó, e implementó, un servicio capaz de interpretar los mensajes de los dispositivos y evaluar el estado de la aplicación; al igual que una proceso de identificar los problemas y definir las acciones a tomar; cumpliendo con el objetivo de determinar una manera por la cual se pudieran realizar dichas comparaciones.

Así mismo, se diseñaron mecanismos que dieran la capacidad de modificar la arquitectura, hacia el estado de referencia definido; cumpliendo con el objetivo de reducir la cantidad de diferencias entre el estado observado, y el deseado.

Finalmente, se validaron las implementaciones realizadas, al igual que se identificaron las falencias de las mismas, dando paso a un nuevo foco de trabajo, ajustando, afinando y extendiendo las maneras en las que se pueden adaptar las arquitecturas de manera autonómica, al igual que estandarizando y simplificando la manera en la que se trabajan con estas herramientas.

De esta primera implementación, rara vez se sale de una línea de comando, por lo que el desarrollo de interfases para los diversos servicios implementados, con el fin de facilitar la accesibilidad a la plataforma.

Así mismo, la extensión de manera de evaluar los estados del sistema, a partir de diferentes datos definidos por el usuario, como uso de promedios para flexibilizar las fallas o contadores, antes de tomar acciones para ignorar fallos temporales, son puntos importantes a mejorar en futuras versiones.

También se ha de considerar la reimplementación de algunos de los mecanismos, con el fin de aumentar su eficacia en modificar los estados. Al igual que la expansión de la base de conocimiento, con el fin de aprovechar todos los recursos disponibles, independiente del estado de algunos componentes.

Se recomienda el buscar formas de permitir, al usuario, definir procesamientos de datos extra para la evaluación de el estado de referencia. Idealmente de manera agnóstica al lenguaje de programación usado. Lo cual daría un valor extra a la flexibilidad de la plataforma.

Finalmente, se recomienda la realización de pruebas más extensas, con dispositivos no simulados, que permitan evolucionar este primer prototipo de plugin para Smart Campus UIS, a algo que se pueda desplegar con relativa facilidad, independientemente del la aplicación que se esté desarrollando.

Referencias

- Alpay, A., y Heuveline, V. (2021, abril). hipSYCL in 2021: Peculiarities, unique features and SYCL 2020. En *International Workshop on OpenCL* (pp. 1–1). Munich Germany: ACM. Descargado 2025-10-09, de <https://dl.acm.org/doi/10.1145/3456669.3456691> doi: 10.1145/3456669.3456691
- Arcaini, P., Riccobene, E., y Scandurra, P. (2015). Modeling and analyzing mape-k feedback loops for self-adaptation. En *2015 ieee/acm 10th international symposium on software engineering for adaptive and self-managing systems* (p. 13-23). doi: 10.1109/SEAMS.2015.10
- Ashraf, Q. M., Tahir, M., Habaebi, M. H., y Isoaho, J. (2023). Toward autonomic internet of things: Recent advances, evaluation criteria, and future research directions. *IEEE Internet of Things Journal*, 10(16), 14725-14748. doi: 10.1109/JIOT.2023.3285359
- Butcher, J. C. (2008). *Numerical methods for ordinary differential equations* (2. Aufl ed.). Chicester: Wiley.
- Camp, D., Garth, C., Childs, H., Pugmire, D., y Joy, K. (2011, noviembre). Streamline Integration Using MPI-Hybrid Parallelism on a Large Multicore Architecture. *IEEE Transactions on Visualization and Computer Graphics*, 17(11), 1702–1713. Descargado 2025-09-10, de <http://ieeexplore.ieee.org/document/5669297/> doi: 10.1109/TVCG.2010.259
- Camp, D., Krishnan, H., Pugmire, D., Garth, C., Johnson, I., Bethel, E. W., ... Childs, H. (2013). *GPU Acceleration of Particle Advection Workloads in a Parallel, Distributed Memory Setting*. The Eurographics Association. Descargado 2025-09-10, de <http://diglib.eg.org/handle/10.2312/EGPGV.EGPGV13.001-008> doi: 10.2312/EGPGV/EGPGV13/001-008
- Docker Inc. (2023). *Develop with docker engine api*. Online. Docker. Descargado de <https://docs.docker.com/engine/api/>

- Durán-Polanco, L., y Siller, M. (2023). A taxonomy for decision making in iot systems. *Internet of Things*, 24, 100904. Descargado de <https://www.sciencedirect.com/science/article/pii/S2542660523002275> doi: <https://doi.org/10.1016/j.iot.2023.100904>
- Eijkhout, V. (2022). *The art of high performance computing: the science of computing. Volume 1* / (3rd edition ed.). North Carolina: Lulu. (OCLC: 1472910278)
- Gorla, A., Pezzè, M., Wuttke, J., Mariani, L., y Pastore, F. (2010, 01). Achieving cost-effective software reliability through self-healing. *Computing and Informatics*, 29, 93-115.
- Gramma, A. (Ed.). (2011). *Introduction to parallel computing* (2. ed., [reprint.] ed.). Harlow: Pearson.
- Hansen, C. D., y Johnson, C. R. (2005). *The visualization handbook*. Amsterdam Boston: Elsevier-Butterworth Heinemann.
- Horn, P. (2001, Oct). *autonomic computing: Ibm's perspective on the state of information technology*. IBM. Descargado de https://homeostasis.scs.carleton.ca/~soma/biosec/readings/autonomic_computing.pdf
- Huynh, N.-T. (2019). An analysis view of component-based software architecture reconfiguration. En *2019 ieee-rivf international conference on computing and communication technologies (rivf)* (p. 1-6). doi: 10.1109/RIVF.2019.8713678
- Iancu, B., y Gatea, A. (2022). Towards a self-describing gateway-based iot solution. En *2022 ieee international conference on automation, quality and testing, robotics (aqtr)* (p. 1-5). doi: 10.1109/AQTR55203.2022.9801938
- Intel Technologies. (2025, marzo). *Intel® oneAPI Programming Guide*. Intel Technologies. Descargado de <https://www.intel.com/content/www/us/en/docs/oneapi/programming-guide/2025-1/overview.html>
- Intel Technologies. (2026, enero). *Intel® oneAPI DPC++/C++ Compiler Release Notes*. Descargado de <https://www.intel.com/content/www/us/en/developer/articles/>

- release-notes/oneapi-dpcpp/2025.html
- Jiménez Herrera, H. A. (2022). *Mecanismos autónomos para la autodescripción de arquitecturas iot distribuidas*. Descargado de <https://noesis.uis.edu.co/items/c5d1cce7-3f8f-4e3e-a083-66bd3c4745f3>
- Jiménez Herrera, H. A. (2023). *Smart campus uis production*. Online. GitHub. Descargado de https://github.com/UIS-IoT-Smart-Campus/smart_campus_production
- Kabashkin, I. (2017). Dynamic reconfiguration of architecture in the communication network of air traffic management system. En *2017 ieee international conference on computer and information technology (cit)* (p. 345-350). doi: 10.1109/CIT.2017.13
- Keryell, R. (2019, noviembre). *SYCL A Single-Source C++ Standard for Heterogeneous Computing*. San José, California. Descargado de https://www.khronos.org/assets/uploads/developers/presentations/SC19-H2RC-keynote-SYCL_Nov19.pdf
- Krikava, F. (2013, 11). *Domain-specific modeling language for self-adaptive software system architectures*.
- Lalanda, P., Diaconescu, A., y McCann, J. A. (2014). *Autonomic computing: Principles, design and implementation*. Springer.
- Laramée, R. S., Hauser, H., Doleisch, H., Vrolijk, B., Post, F. H., y Weiskopf, D. (2004, junio). The State of the Art in Flow Visualization: Dense and Texture-Based Techniques. *Computer Graphics Forum*, 23(2), 203–221. Descargado 2025-09-30, de <https://onlinelibrary.wiley.com/doi/10.1111/j.1467-8659.2004.00753.x> doi: 10.1111/j.1467-8659.2004.00753.x
- Liu, H., Parashar, M., y Hariri, S. (2004). A component-based programming model for autonomic applications. En *International conference on autonomic computing, 2004. proceedings*. (p. 10-17). doi: 10.1109/ICAC.2004.1301341
- Mittal, S., y Vetter, J. S. (2015, enero). A Survey of Methods for Analyzing and Improving GPU Energy Efficiency. *ACM Computing Surveys*, 47(2), 1–23. Descargado 2025-09-30, de <https://dl.acm.org/doi/10.1145/2636342> doi: 10.1145/2636342

- Ouareth, S., Boulehouache, S., y Mazouzi, S. (2018). A component-based mape-k control loop model for self-adaptation. En *2018 3rd international conference on pattern analysis and intelligent systems (pais)* (p. 1-7). doi: 10.1109/PAIS.2018.8598529
- Patouni, E., y Alonistioti, N. (2006). A framework for the deployment of self-managing and self-configuring components in autonomic environments. En *2006 international symposium on a world of wireless, mobile and multimedia networks(wowmom'06)* (p. 5 pp.-484). doi: 10.1109/WOWMOM.2006.11
- Post, F. H., Vrolijk, B., Hauser, H., Laramée, R. S., y Doleisch, H. (2003, diciembre). The State of the Art in Flow Visualisation: Feature Extraction and Tracking. *Computer Graphics Forum*, 22(4), 775–792. Descargado 2025-09-30, de <https://onlinelibrary.wiley.com/doi/10.1111/j.1467-8659.2003.00723.x> doi: 10.1111/j.1467-8659.2003.00723.x
- Pugmire, D., Childs, H., Garth, C., Ahern, S., y Weber, G. H. (2009, noviembre). Scalable computation of streamlines on very large datasets. En *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis* (pp. 1–12). Portland Oregon: ACM. Descargado 2025-09-30, de <https://dl.acm.org/doi/10.1145/1654059.1654076> doi: 10.1145/1654059.1654076
- Rajan, M., Balamuralidhar, P., Chethan, K., y Swarnahpriyaah, M. (2011). A self-reconfigurable sensor network management system for internet of things paradigm [Conference paper]. Descargado de <https://www.scopus.com/inward/record.uri?eid=2-s2.0-84858147900&doi=10.1109%2fICDECOM.2011.5738550&partnerID=40&md5=ed1f7dcccdf65e520a328eb3a19b581e> (Cited by: 17) doi: 10.1109/ICDECOM.2011.5738550
- Reinders, J., Ashbaugh, B., Brodman, J., Kinsner, M., Pennycook, J., y Tian, X. (2021). *Data Parallel C++: Mastering DPC++ for Programming of Heterogeneous Systems using C++ and SYCL*. Berkeley, CA: Apress. Descargado 2025-10-07, de <http://link.springer.com/10.1007/978-1-4842-5574-2> doi: 10.1007/978-1-4842-5574-2

- Salehie, M., y Tahvildari, L. (2005, 01). Autonomic computing: emerging trends and open problems. *ACM SIGSOFT Software Engineering Notes*, 30, 1-7.
- Shvets, A. (2019). *Dive into design patterns* (R. S. Andrew Wetmore, Ed.). Refactoring Guru.
- Süli, E., y Mayers, D. F. (2003). *An Introduction to Numerical Analysis* (1.^a ed.). Cambridge University Press. Descargado 2025-09-30, de <https://www.cambridge.org/core/product/identifier/9780511801181/type/book> doi: 10.1017/CBO9780511801181
- The Khronos Group Inc. (2025, abril). *SYCL™ 2020 Specification (revision 10)*. The Khronos Group Inc. Descargado de <https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html>
- Wang, Z., Sun, C.-a., y Aiello, M. (2021). Lightweight and context-aware modeling of microservice-based internet of things. En *2021 ieee international conference on web services (icws)* (p. 282-292). doi: 10.1109/ICWS53863.2021.00046
- Weiss, G., Zeller, M., y Eilers, D. (2011). *Towards automotive embedded systems with self-x properties*. Fraunhofer-Gesellschaft. Descargado de <https://publica.fraunhofer.de/handle/publica/224379> doi: 10.24406/PUBLICA-FHG-224379

Apéndices

1 Directivas declaradas la aplicación Malaga

```
1 application:
2   name: Malaga
3   description: "Test Scenario A"
4
5 locations:
6   malaga:
7     name: "Sede Malaga"
8     locations:
9       north-sector:
10        name: "Sector Norte"
11        data-requirements:
12          temperature:
13            output: number
14            required: true
15            count: 1
16            timeout: 30
17          wind-speed:
18            output: number
19            required: true
20            count: 3
21            timeout: 60
22          particulate-matter:
23            output: number
24            required: false
25            count: 2
26            timeout: 120
27        south-sector:
28          name: "Sector Sur"
29          data-requirements:
30            wind-speed:
31              output: number
32              required: true
33              count: 3
34              timeout: 60
35            particulate-matter:
36              output: number
37              required: false
38              count: 2
39              timeout: 120
40            temperature:
41              output: number
42              required: true
43              count: 1
44              timeout: 30
```

2 Directivas de la aplicación Malaga

```

1 {
2   "north-sector": {
3     "addition": {
4       "image": "mrdahaniel/mocker:latest",
5       "network_name": "smart_campus_production_smart_uis",
6       "env_vars": {
7         "RUST_LOG": "mockerr",
8         "ip": "mqtt-broker"
9       },
10      "args": [
11        "key:random(5,35) "
12      ],
13    },
14    "reconfig": {
15      "uuid": null,
16      "network": "smart_campus_production_smart_uis",
17      "query_type": {
18        "Http": {
19          "endpoint": "/get/device",
20          "port": 8000
21        }
22      },
23      "reconfig": {
24        "Http": {
25          "endpoint": "/update/interval",
26          "port": 8000,
27          "method": "PUT",
28          "payload": {
29            "nanos": 0,
30            "secs": 25
31          }
32        }
33      }
34    },
35    "restart": {
36      "uuid": null,
37      "query_type": {
38        "Http": {
39          "endpoint": "/get/device",
40          "port": 8000
41        }
42      }
43    }
44  },
45  "south-sector": {
46    "addition": {
47      "image": "mrdahaniel/mocker:latest",
48      "network_name": "smart_campus_production_smart_uis",
49      "env_vars": {
50        "ip": "mqtt-broker",
51        "RUST_LOG": "mockerr"
52      },
53      "args": [
54        "key:random(5,35) "
55      ],
56    },
57    "reconfig": {
58      "uuid": null,
59      "network": "smart_campus_production_smart_uis",
60      "query_type": {
61        "Http": {
62          "endpoint": "/get/device",
63          "port": 8000
64        }
65      },
66      "reconfig": {
67        "Http": {

```

```
68         "endpoint": "/update/interval",
69         "port": 8000,
70         "method": "PUT",
71         "payload": {
72             "nanos": 0,
73             "secs": 25
74         }
75     },
76 },
77 },
78 "restart": {
79     "uuid": null,
80     "query_type": {
81         "Http": {
82             "endpoint": "/get/device",
83             "port": 8000
84         }
85     }
86 },
87 },
88 }
```

3 YAML de la aplicación Patio

```
1 application:
2   name: Patio
3   description: "Patio App"
4
5 locations:
6   greenhouse:
7     name: "Greenhouse"
8     data-requirements:
9       temperature:
10        output: number
11        required: true
12        count: 1
13        timeout: 30
14     co2:
15       output: number
16       required: true
17       count: 1
18       timeout: 120
19     humidity:
20       output: number
21       required: true
22       count: 1
23       timeout: 120
```

4 YAML la aplicación House

```
1 application:
2   name: House
3   description: "House App"
4
5 locations:
6   first-floor:
7     name: "First Floor"
8     data-requirements:
9       temperature:
10        output: number
11        required: true
12        count: 1
13        timeout: 30
14   basement:
15     name: "Basement"
16     data-requirements:
17       co:
18        output: number
19        required: false
20        count: 1
21        timeout: 30
```

5 Directivas de la aplicación Patio

```

1 {
2   "greenhouse": {
3     "addition": {
4       "image": "mrdahaniel/mocker:latest",
5       "network_name": "smart_campus_production_smart_uis",
6       "env_vars": {
7         "RUST_LOG": "mock",
8         "ip": "mqtt_broker"
9       },
10      "args": [
11        "key:random(5,35)"
12      ]
13    },
14    "reconfig": {
15      "uuid": null,
16      "network": "smart_campus_production_smart_uis",
17      "query_type": {
18        "Http": {
19          "endpoint": "/get/device",
20          "port": 8000
21        }
22      },
23      "reconfig": {
24        "Http": {
25          "endpoint": "/update/interval",
26          "port": 8000,
27          "method": "PUT",
28          "payload": {
29            "nanos": 0,
30            "secs": 25
31          }
32        }
33      }
34    },
35    "restart": {
36      "uuid": null,
37      "query_type": {
38        "Http": {
39          "endpoint": "/get/device",
40          "port": 8000
41        }
42      }
43    }
44  }
45 }

```

6 Directivas de la aplicación House

```

1 {
2   "basement": {
3     "addition": {
4       "image": "mrdahaniel/mocker:latest",
5       "network_name": "smart_campus_production_smart_uis",
6       "env_vars": {
7         "RUST_LOG": "mock",
8         "ip": "mqtt_broker"
9       },
10      "args": [
11        "key:random(5,35)"
12      ]
13    },
14    "reconfig": {
15      "uuid": null,
16      "network": "smart_campus_production_smart_uis",
17      "query_type": {
18        "Http": {
19          "endpoint": "/get/device",
20          "port": 8000
21        }
22      },
23      "reconfig": {
24        "Http": {
25          "endpoint": "/update/interval",
26          "port": 8000,
27          "method": "PUT",
28          "payload": {
29            "nanos": 0,
30            "secs": 25
31          }
32        }
33      }
34    },
35    "restart": {
36      "uuid": null,
37      "query_type": {
38        "Http": {
39          "endpoint": "/get/device",
40          "port": 8000
41        }
42      }
43    }
44  },
45  "first-floor": {
46    "addition": {
47      "image": "mrdahaniel/mocker:latest",
48      "network_name": "smart_campus_production_smart_uis",
49      "env_vars": {
50        "ip": "mqtt_broker",
51        "RUST_LOG": "mock"
52      },
53      "args": [
54        "key:random(5,35)"
55      ]
56    },
57    "reconfig": {
58      "uuid": null,
59      "network": "smart_campus_production_smart_uis",
60      "query_type": {
61        "Http": {
62          "endpoint": "/get/device",
63          "port": 8000
64        }
65      },
66      "reconfig": {
67        "Http": {

```

```
68         "endpoint": "/update/interval",
69         "port": 8000,
70         "method": "PUT",
71         "payload": {
72             "nanos": 0,
73             "secs": 25
74         }
75     }
76 }
77 },
78 "restart": {
79     "uuid": null,
80     "query_type": {
81         "Http": {
82             "endpoint": "/get/device",
83             "port": 8000
84         }
85     }
86 }
87 }
88 }
```
